

Ministère de
l'Enseignement Supérieur
et de la Recherche
Scientifique
Université de Carthage
Ecole Polytechnique de
Tunisie

وزارة التعليم العالي و
البحث العلمي
جامعة قرطاج
المدرسة التونسية للتقنيات

Graduation Project Report

To obtain :

The National Engineering Diploma from the Ecole Polytechnique
de Tunisie

Compositional Difficulty in Multi-Hop Question Answering

Elaborated by:

Omar Abdelhedi

Within:



Supervised by:

Dr. Amor MESSAOUD

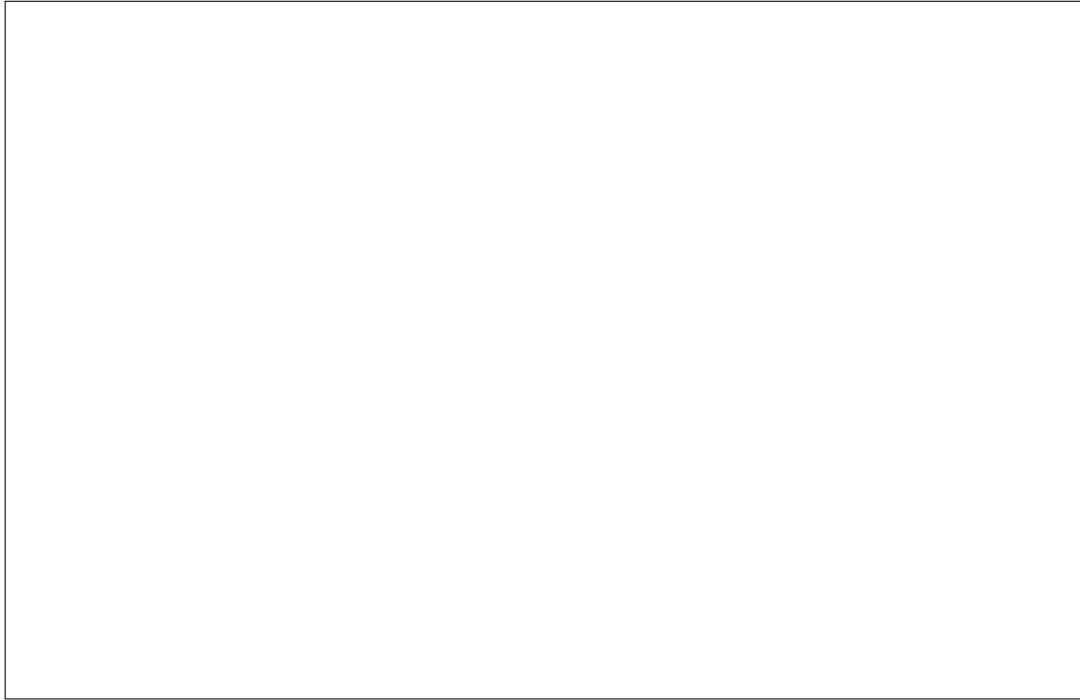
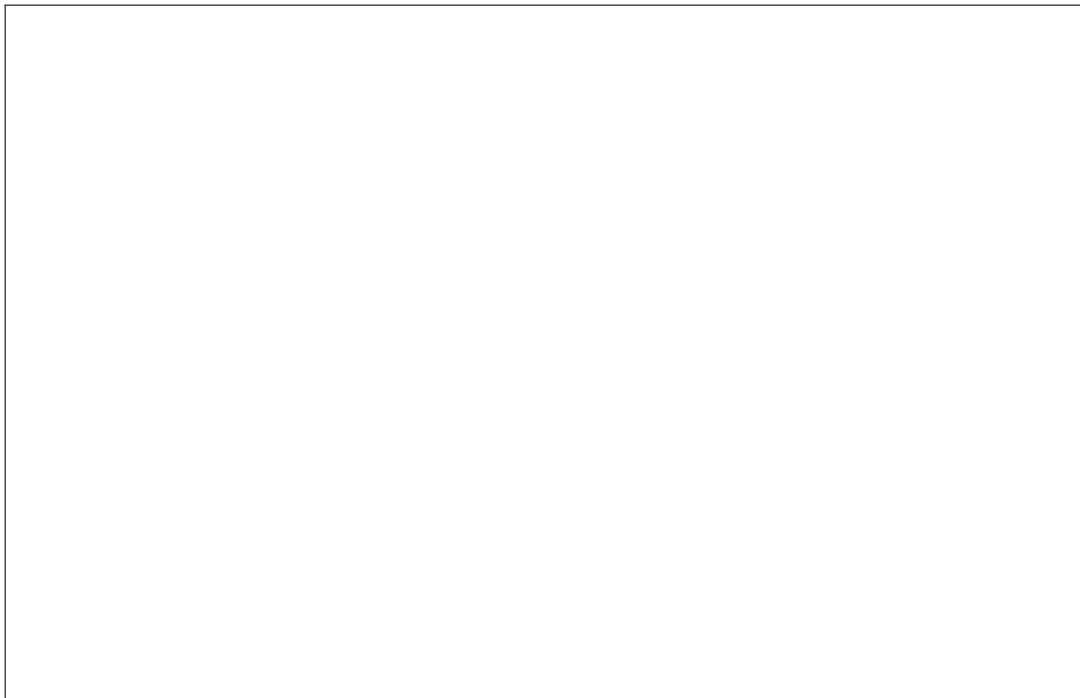
Academic Supervisor

Dr. Yassine YAAKOUBI

Professional Supervisor

Academic year: 2025/2026

Assessments and signature of the supervisors

A large, empty rectangular box with a thin black border, intended for supervisor assessments and signature.A second large, empty rectangular box with a thin black border, identical to the one above, for supervisor assessments and signature.

Abstract

Keywords:

Résumé

Mots clés :

Dedications

Acknowledgements

Contents

Abstract / Résumé	ii
Dedications	iii
Acknowledgements	iv
List of Figures	ix
List of Tables	x
List of Abbreviations	xii
 General Introduction	 1
1 General Presentation and Context of the Internship	2
Introduction	2
1.1 Host organisation	2
1.1.1 Concordia University	3
1.1.2 Gina Cody School of Engineering and Computer Science	3
1.1.3 Research group, supervision, and research axes	4
1.2 Problem statement	5
1.3 Objectives and expected deliverables	5
1.4 Working methodology	5
Conclusion	5
 2 Background and State of the Art	 6
Introduction	6
2.1 Multi-Hop Question Answering and Retrieval-Augmented Generation	6
2.1.1 Retrieval-Augmented Generation	6
2.1.2 Multi-Hop Question Answering	7
2.1.3 Multi-Hop Question Answering Benchmarks	7
2.1.4 Evidence and Supporting Passages	8
2.1.5 Evidence Coverage and Retrieval Depth	9
2.2 Retrieval for Multi-Hop Reasoning	10
2.2.1 Sparse Retrieval	10
2.2.2 Dense Retrieval	11
2.2.3 Late-Interaction Retrieval and Reranking	11
2.2.4 Iterative and Multi-Hop Retrieval	11

2.2.5	Graph-Based Retrieval for Multi-Hop Reasoning	12
2.3	Difficulty-Aware and Adaptive Computation	12
2.4	Probing Internal Representations of Language Models	13
2.4.1	Linear Probing and Representation Analysis	14
2.4.2	Latent Properties in LLM Representations	14
2.5	Question Decomposition for Multi-Hop Reasoning	15
2.5.1	Why Decomposition Helps Multi-Hop Reasoning	15
2.5.2	Earlier Question-Decomposition Approaches	16
2.5.3	LLM-Based Decomposition	16
2.6	Research Gap and Thesis Positioning	17
2.6.1	Anticipating Compositional Difficulty	17
2.6.2	Optimising Decomposition for Retrieval Efficiency	17
	Conclusion	17
3	Probing Compositional Question Difficulty in Language Model Representations	19
	Introduction	19
3.1	Research Question and Problem Formulation	19
3.2	Methodology	20
3.2.1	Hidden-State Extraction and Question Representation	20
3.2.2	Linear Probe and Layer-Wise Evaluation	21
3.2.3	Label-Permutation Null	22
3.2.4	Surface-Feature Control and Residualisation	22
3.2.5	Ordinality Test	23
3.2.6	Cross-Dataset Transfer	23
3.3	Experimental Setup	24
3.3.1	Language Models	24
3.3.2	Benchmarks and Difficulty Labels	24
3.3.3	Evaluation Configuration	24
3.4	Results	24
3.4.1	Layer-Wise Decodability of Compositional Difficulty	24
3.4.2	Robustness Beyond Surface Features	25
3.4.3	Ordinal Structure of the Difficulty Direction	27
3.4.4	Cross-Dataset Transfer of the Residualised Difficulty Direction	28
3.5	Discussion and Limitations	29
3.5.1	Interpretation and Implications for Adaptive Computation	29
3.5.2	Limitations and Future Directions	30
	Conclusion	31
4	Evidence-Aware Evaluation of Question Decomposition	32
4.1	Problem Formulation	32
4.1.1	A Decomposition as an Executable Sequence	32
4.1.2	Availability, Assignment and Execution	33
4.2	Retrieval Depth and Evidence Responsibility	33
4.2.1	The Evidence-Rank Matrix	33
4.2.2	Union Depth	34
4.2.3	Ownership-Masked Depth	34
4.2.4	Coverage and an Aggregate Depth Score	35
4.3	Learning the Ownership Mask	35

4.3.1	Supervision from Reference Decompositions	35
4.3.2	Textual and Contextual Features	36
4.3.3	Fitting, Threshold Selection and Ensemble Decision	36
4.3.4	How Assignment Error Changes the Depth Measurement	37
4.4	Experimental Protocol	37
4.4.1	Systems and Retained Executions	37
4.4.2	Metrics and Comparison Units	37
4.5	Results	38
4.5.1	Ownership Recovery on Reference-Style Arms	38
4.5.2	Sensitivity to the Attribution Model	39
4.5.3	Evidence Availability and Final Answers	39
4.5.4	The Role of Intermediate Bindings	40
4.6	Depth Under a Total Passage Budget	41
4.6.1	Equal-Allocation Replay	41
4.6.2	A Fixed-List Oracle Allocation	42
4.6.3	Allocation Headroom on the Recorded Lists	42
4.7	A Cross-Benchmark Fixed-Trace Study	43
4.7.1	Retrieval Regime and Evaluation Population	43
4.7.2	Matched Passage-Slot Frontiers	43
4.7.3	Reference and Generated Query Regimes	44
4.8	Discussion and Implications for Optimisation	45
5	Optimising Question Decomposition and Evidence Allocation	47
5.1	The Optimisation Problem	48
5.1.1	An Instruction Defines an Executable Policy	48
5.1.2	The Retained Retrieval Objective	48
5.1.3	Answer Quality and Resource Use	49
5.2	The Completed OPRO Search	49
5.2.1	Search Configuration	49
5.2.2	Proposal and Selection Loop	50
5.2.3	The Feedback Supplied to the Optimizer	50
5.3	Results of the Pilot	50
5.3.1	Progress Across Search Rounds	50
5.3.2	The Selected Instruction	51
5.3.3	The Optimisation Objective and the Answer Objective	51
5.4	Learned Allocation on Fixed Retrieval Traces	52
5.4.1	The Allocation Intervention	52
5.4.2	Predicting Per-Arm Depth Requirements	53
5.4.3	Segment and Unit-Increment Allocation	53
5.4.4	Evaluation Population and Cross-Validation	54
5.4.5	Coverage and Target Budgets	55
5.4.6	Paired Final-Answer Evaluation	55
5.4.7	Exploratory Hidden-State Prediction of Assigned Depth	56
5.5	Evidence-Aware Feedback for Prompt Search	57
5.5.1	Separating Reward from Diagnostic Information	57
5.5.2	Observed Events and Estimated Assignments	58
5.5.3	Reflective Search with a Fixed Execution Interface	58
5.6	Independent Evaluation Design	59

5.6.1	Frozen Data Roles and Corpus Scope	59
5.6.2	Comparators and Search Allowances	60
5.6.3	Outcome Analysis and Cost Accounting	61
5.6.4	Separating Prompt and Allocation Effects	61
5.6.5	Controlled Checks of the Diagnostic Representation	62
5.7	Discussion	62
5.7.1	What the Completed Experiment Establishes	62
5.7.2	The Scope of the Evidence-Feedback Extension	63
Conclusion and outlook		65
Bibliography		66
Netography		69

List of Figures

1.1	The EV Complex, home of the Gina Cody School (photograph: Tasitch, CC BY 2.0)	4
2.1	Separation of retrieval and generation in RAG	7
2.2	Multi-hop reasoning through a bridge entity	8
2.3	Per-question retrieval depth and cohort coverage	9
3.1	Overview of the compositional-difficulty probing protocol	20
3.2	Difficulty decoding across model depth	26
3.3	Label-permutation null across depth	26
3.4	Raw, surface-only, and residualised difficulty decoding	27
3.5	Ordinality of the decoded MuSiQue difficulty direction	28
3.6	Cross-benchmark transfer to MuSiQue	28
4.1	Execution conditions in the retrieval study. The diagram is the execution shared by every condition, with the two marked points that the comparison varies: the source of the subquestion sequence and the source of the intermediate answers substituted into later subquestions. The table states what each condition supplies at those two points. Direct execution has neither, and the gold-binding condition is oracle-assisted.	38

List of Tables

1.1	Host organisation details	3
3.1	Layer-wise difficulty decoding and surface controls	25
3.2	Leave-one-benchmark-out transfer of the difficulty axis	29
4.1	Ownership prediction on the reference-style decomposition control. Cell metrics use 814 subquestion–passage pairs. Complete-mask agreement is measured over the 100 questions.	39
4.2	Ownership errors by reference hop count. TP, FN, FP and TN count individual cells, while n counts questions.	39
4.3	Masked-depth measurements under two attribution models on fixed GPT-4o-mini retrieval traces. Coverage is complete assigned-evidence coverage at $K = 200$; score is J_{D_M} from Equation 4.9. The last row uses the exact reference assignment.	40
4.4	Recorded execution results on 100 MuSiQue questions. EM and F1 are percentages. $C_U(5)$ is complete union coverage at five passages per arm. Arms and tokens are per-question means. Retrieved evidence and final answers are separate outcomes.	40
4.5	Reference-decomposition controls with GPT-4o-mini. Gold bindings supply reference intermediate answers when constructing downstream queries. Predicted bindings use the reader’s earlier outputs.	41
4.6	Complete evidence coverage (%) at a total budget of 15 passage slots on fixed recorded lists. The oracle chooses prefixes using gold evidence and is a reference allocation rather than a deployed policy. Each row contains 100 questions.	42
4.7	Complete annotated-evidence coverage at a ceiling of eight selected passage slots. BM25 results use fixed predicted-binding traces; the oracle uses gold evidence ranks. Counts are retained questions from each original 1,000-question panel.	44
4.8	MuSiQue BM25 retrieval regimes. Coverage is measured on each retained panel under balanced slot allocation. The reference row uses a different plan source and a larger panel.	44
5.1	Configuration of the completed prompt-search pilot.	49
5.2	Masked-depth score during the completed search. The retained best includes all earlier rounds. Values are on the shared development panel. . . .	51
5.3	Initial and selected instructions on the 100 development questions. F1 is displayed on a 0–100 scale. Passage slots are the five-per-arm allowance derived from the observed arm count.	51

5.4	Different objectives select different candidates from the same development search. The F1-leading candidate is identified retrospectively.	52
5.5	Allocation on fixed BM25 traces. Segment denotes the segment heuristic and Unit the unit-increment optimiser of the concave interpolant. The budget counts selected slots, excluding trace generation and shallow-score acquisition. Differences are percentage points relative to balanced allocation.	55
5.6	Smallest tested passage-slot budget reaching a coverage target on the fixed generated sequential traces, using the segment policy. The oracle uses gold passage ranks and permits zero-depth arms. Ratios compare discrete operating points.	55
5.7	Pooled final-reader evaluation on 700 retained MuSiQue traces. U denotes balanced allocation and L the learned segment heuristic. All 4,200 question–budget–policy evaluations completed. The last column gives L minus U and its descriptive pointwise paired 95% F1 interval; the intervals at budgets 16 and 32 contain zero. The unit-increment allocator is a separate policy.	56
5.8	Exploratory depth prediction on the fixed reference panel. Mean AUROC averages the five depth thresholds. Each model’s layer is selected using the same cross-validation results reported here.	57
5.9	Diagnostic fields supplied by the structured feedback interface.	58
5.10	Frozen question-new splits for the prospective comparison. Components connect questions sharing a source single-hop ID.	60
5.11	Ownership predictions on the optimisation-only stress suite. Exact agreement requires every cell of a case’s ownership mask to match the transformation labels. Variants share twelve base questions.	62

List of Abbreviations

AI	Artificial Intelligence
AI4DM	AI for Decision Making Lab, Concordia University
AUC	Area Under the Receiver Operating Characteristic Curve
BM25	Best Match 25, a probabilistic lexical ranking function
ColBERT	Contextualized Late Interaction over BERT
DAG	Directed Acyclic Graph
DPR	Dense Passage Retrieval
EPT	Tunisia Polytechnic School (<i>École Polytechnique de Tunisie</i>)
EV	Engineering, Computer Science and Visual Arts Complex, Concordia University
FLARE	Forward-Looking Active Retrieval Augmented Generation
IRCoT	Interleaving Retrieval with Chain-of-Thought Reasoning
LDA	Linear Discriminant Analysis
LLM	Large Language Model
MDR	Multi-Hop Dense Retrieval
MIAE	Mechanical, Industrial and Aerospace Engineering, Concordia University
RAG	Retrieval-Augmented Generation
ROC	Receiver Operating Characteristic
TF-IDF	Term Frequency-Inverse Document Frequency

General Introduction

Chapter 1

General Presentation and Context of the Internship

Introduction

The purpose of this chapter is to situate the project within its general research and institutional context, and to present the main objectives, research directions, and methodology that guided the work carried out during the internship.

The work reported in this thesis was conducted as a research internship at the AI for Decision Making Lab of Concordia University, under the supervision of Dr. Yassine Yaakoubi, and constitutes the final-year project of the engineering curriculum at École Polytechnique de Tunisie (EPT).

This chapter first introduces the host institution and the research laboratory in which the internship was carried out, together with their main areas of research. It then presents the broader scientific context of the internship, centered on large language models, retrieval-augmented generation, and multi-hop question answering. The progression of the research work is subsequently outlined, from the initial literature exploration and systematic study of trust signals in RAG systems, to the investigation of compositional question difficulty in language model representations, and finally to the development of a retrieval-oriented framework for multi-hop question decomposition. The chapter concludes by presenting the main objectives, contributions, and research methodology adopted throughout the internship.

1.1 Host organisation

The internship was hosted by the **AI for Decision Making Lab (AI4DM)**, a research group within the Department of Mechanical, Industrial and Aerospace Engineering (MIAE) of the Gina Cody School of Engineering and Computer Science at Concordia University, Montreal, Canada. Table 1.1 summarises the host organisation. The following subsections present the university, the faculty and department to which the laboratory belongs, and finally the research group in which the work was carried out.

Table 1.1: Host organisation details

Institution	Concordia University
Faculty	Gina Cody School of Engineering and Computer Science
Department	Mechanical, Industrial and Aerospace Engineering (MIAE)
Laboratory	AI for Decision Making Lab (AI4DM)
Supervisor	Dr. Yassine Yaakoubi
Location	Montreal, Quebec, Canada
Research axes	Optimisation, learning and simulation for large-scale decision making under uncertainty; large language models and retrieval-augmented generation; uncertainty quantification and reliability of AI systems

1.1.1 Concordia University

Concordia University is a public comprehensive research university located in Montreal, Quebec. It was officially founded in 1974 following the merger of Loyola College, established in 1896, and Sir George Williams University, whose origins can be traced to the educational activities of the Montreal YMCA [N1, N2]. The name of the university derives from the motto of the City of Montreal, *Concordia salus*, meaning “well-being through harmony.”

Concordia operates two main campuses: the Sir George Williams Campus in downtown Montreal and the Loyola Campus in the west of the city. In the 2024–2025 academic year, 43,922 students were enrolled in credit courses across the university [N3]. Its academic activities cover a broad range of disciplines through the Faculty of Arts and Science, the Gina Cody School of Engineering and Computer Science, the John Molson School of Business, and the Faculty of Fine Arts.

1.1.2 Gina Cody School of Engineering and Computer Science

The Gina Cody School of Engineering and Computer Science is Concordia University’s faculty dedicated to engineering and computer science. In 2018, following a fifteen-million-dollar donation from Concordia alumna Gina Parvaneh Cody, the faculty was renamed in her honour, becoming the first engineering faculty in Canada to be named after a woman [N4]. Gina Cody, who earned her doctorate in building engineering from Concordia in 1989 and was the first woman in Canada to obtain a PhD in the discipline, has served as Chancellor of Concordia University since January 2025 [N5].

The school brings together 11,315 undergraduate and graduate students, 262 faculty members, 43 undergraduate and graduate programmes, and 20 interdisciplinary research centres [N6]. Its academic structure comprises seven departments, including the Department of Mechanical, Industrial and Aerospace Engineering (MIAE), to which the host research group belongs.

Teaching and research activities are conducted principally in the Engineering, Computer Science and Visual Arts Integrated Complex (EV Building), located on Concordia’s Sir George Williams Campus and shown in Figure 1.1.



Figure 1.1: The EV Complex, home of the Gina Cody School (photograph: Tasitch, CC BY 2.0)

1.1.3 Research group, supervision, and research axes

The work presented in this report was conducted within the AI for Decision Making Lab (AI4DM), led by Dr. Yassine Yaakoubi, Assistant Professor in the Department of Mechanical, Industrial and Aerospace Engineering at Concordia University.

The research activities of the group lie at the intersection of optimisation, machine learning, and simulation for large-scale real-world decision making. The group combines mathematical optimisation methods, including mixed-integer and stochastic programming and decomposition techniques, with machine-learning approaches such as deep learning, reinforcement learning, and graph-based models. The general objective is to design decision-support systems that remain efficient, robust, and scalable in complex and uncertain environments, with applications including logistics, transportation, supply chains, and sustainability.

A major part of the group's work is organised around machine-learning-augmented optimisation, where learned models are integrated into classical optimisation procedures, and end-to-end optimisation learning, where learning mechanisms are used more directly to guide decision making.

A further strand of the group's research activity extends these concerns with robustness, uncertainty, and adaptive decision making to systems built on large language models. This includes retrieval-augmented generation, where language models are grounded in evidence retrieved from external knowledge sources, as well as uncertainty quantification, reliability assessment, and adaptive behaviour in such systems.

The work carried out during this internship belongs primarily to this latter research direction. It focuses on multi-hop question answering in retrieval-augmented systems and investigates, from complementary perspectives, how the difficulty of such questions can be understood and how their processing can be made more effective. The research therefore evolved from the study of reliability and actionable signals in RAG systems, to the analysis of compositional question difficulty in the internal representations of large

language models, and finally to the development of a retrieval-oriented approach to multi-hop question decomposition.

This research environment provided the basis for the successive studies conducted during the internship and for the formulation of the main research problem addressed in this thesis.

1.2 Problem statement

1.3 Objectives and expected deliverables

1.4 Working methodology

Conclusion

Chapter 2

Background and State of the Art

Introduction

Multi-hop question answering requires a system to retrieve and combine several pieces of evidence before producing an answer. In Retrieval-Augmented Generation (RAG), this creates a particular difficulty: the evidence needed for a complex question may not be easily retrieved from the original query, and different questions may require very different amounts of retrieval and reasoning.

This chapter introduces the concepts needed for the remainder of the thesis. It first presents RAG and multi-hop question answering, then reviews retrieval methods and the notions of evidence coverage and retrieval depth. It subsequently discusses adaptive computation, probing of internal model representations, and question decomposition, before focusing on decomposition as a means of making evidence easier to retrieve. The chapter ends by identifying the two research gaps addressed in the following chapters.

2.1 Multi-Hop Question Answering and Retrieval-Augmented Generation

2.1.1 Retrieval-Augmented Generation

Large Language Models (LLMs) acquire substantial knowledge during pre-training, but their parametric knowledge is neither a complete nor a continuously updated record of the world. It may omit specialised facts, become outdated, or be insufficient when a question requires information from a particular source. RAG complements the model with an external corpus that can be searched at inference time [1].

Let the corpus be

$$\mathcal{C} = \{d_1, d_2, \dots, d_N\}, \quad (2.1)$$

where each d_i is a document or passage. For a query q , a retriever assigns a relevance score $s(q, d)$ and returns

$$D_k(q) = \underset{d \in \mathcal{C}}{\text{TopK}} s(q, d). \quad (2.2)$$

The generator then predicts an answer y conditional on the question and the retrieved passages,

$$p(y \mid q, D_k(q)). \quad (2.3)$$

This division of labour is central: the retriever determines which external information becomes visible, while the generator interprets and combines that information. Figure 2.1

depicts the separation between the two components. A stronger generator can reason more effectively over retrieved context, but it cannot directly exploit a relevant passage that the retriever never exposes.

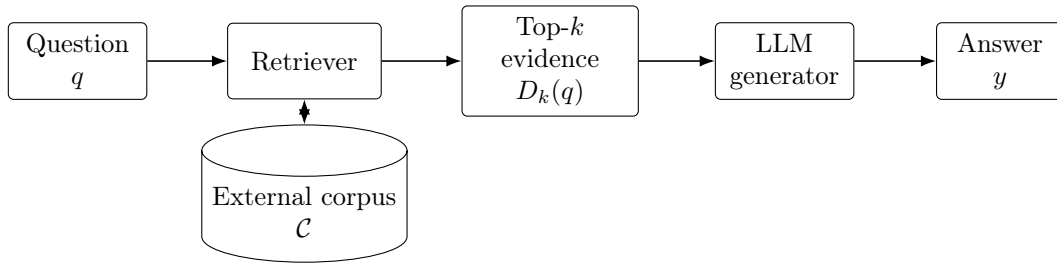


Figure 2.1: A conventional RAG pipeline. Retrieval selects a small portion of the external corpus; only those passages are available to the language model during generation.

The retrieval cutoff k controls both opportunity and cost. A larger context may contain more useful evidence, but also introduces distractors and increases the amount of text processed by the generator. This tension is mild when one passage directly answers the question and more consequential when the answer depends on several passages with different lexical and semantic relations to the original query.

2.1.2 Multi-Hop Question Answering

A single-hop question can normally be resolved through one principal factual relation or evidence source. A multi-hop question instead requires several dependent facts or operations,

$$r_1 \longrightarrow r_2 \longrightarrow \cdots \longrightarrow r_h, \quad (2.4)$$

where an intermediate result may determine what must be resolved next. The entity or value transferred between two steps is called a *bridge entity*. For example, the question *Which country is the birthplace of the author of Book X?* first requires identifying the author, then retrieving that person’s birthplace. Figure 2.2 makes this dependency explicit.

Reasoning hop count and retrieval depth describe different quantities and must not be conflated. A four-hop benchmark question encodes four dependent reasoning operations. Retrieval depth $k = 50$, by contrast, means that up to fifty ranked passages are considered for a query. Later chapters seek to reduce the latter; they do not alter the benchmark’s compositional hop count.

2.1.3 Multi-Hop Question Answering Benchmarks

Multi-hop benchmarks operationalise compositional reasoning in different ways. Three datasets are particularly relevant to this thesis. **HotpotQA** is a Wikipedia-based benchmark designed for diverse, explainable multi-hop question answering. In addition to answers, it annotates supporting facts, enabling joint evaluation of answer prediction and evidence identification [2].

2WikiMultiHopQA constructs multi-hop questions from Wikipedia and Wikidata information and provides explicit evidence and reasoning structures. Its question types include comparison and compositional chains, allowing models to be assessed at intermediate as well as final steps [3].

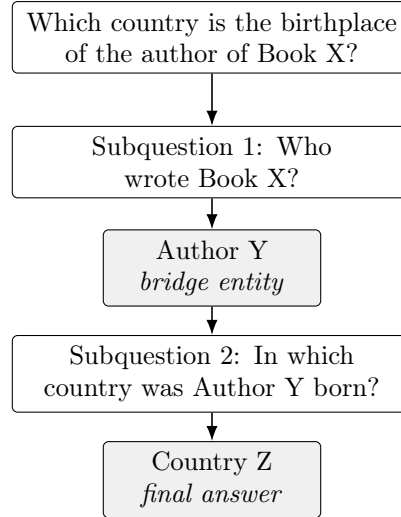


Figure 2.2: A two-hop reasoning chain. The answer to the first subquestion supplies a bridge entity needed to formulate and resolve the second information need.

MuSiQue is constructed bottom-up by composing connected single-hop questions into questions requiring two to four hops. Its construction preserves dependencies among intermediate questions and supporting passages, while filtering examples that can be answered through shortcuts [4]. This structured relation between intermediate questions and evidence is also useful when studying how generated decompositions divide responsibility for retrieving the evidence.

The three benchmarks therefore provide distinct operational views of compositional difficulty. They form the empirical basis of the layer-wise probing study introduced in Chapter 3 [5], without assuming that a nominal hop label is a complete measure of all forms of difficulty.

2.1.4 Evidence and Supporting Passages

Let the gold evidence for a question be

$$E(q) = \{e_1, e_2, \dots, e_m\}, \quad (2.5)$$

and let the retriever produce the ordered ranking

$$R(q) = (d_1, d_2, \dots, d_N). \quad (2.6)$$

For $e \in E(q)$, $\text{rank}(e \mid q)$ denotes its one-based position in this ranking. Evidence retrieval is successful only to the extent that the required supporting passages appear within the part of the ranking made available downstream.

Answer correctness and evidence retrieval success are related but not equivalent. An LLM can occasionally answer correctly from parametric memory even when the annotated supporting passages are absent. Conversely, retrieving the gold passages does not guarantee that the generator will combine them correctly. For this reason, the later evaluation isolates whether required evidence is available instead of inferring retrieval quality solely from the final answer.

2.1.5 Evidence Coverage and Retrieval Depth

A multi-hop question is not answerable from part of its evidence. The quantity of interest is therefore not how much of $E(q)$ a ranking contains, but how deep the ranking must be read before it contains all of it. For a question whose evidence is all present, that depth is fixed by the lowest-ranked required passage:

$$K^*(q) = \max_{e \in E(q)} \text{rank}(e \mid q). \quad (2.7)$$

Equivalently, $K^*(q)$ is the smallest cutoff k at which $E(q) \subseteq D_k(q)$. If any required passage falls outside the evaluated search horizon, $K^*(q)$ is undefined and the question is *censored*: assigning it an artificial finite depth would make a retrieval failure appear efficient.

Depth is a per-question quantity, so a set of questions is summarised by the proportion whose evidence is fully available by a given depth:

$$\text{Coverage}(k) = \frac{|\{q \in Q : K^*(q) \leq k\}|}{|Q|}. \quad (2.8)$$

A value of 0.66 at k means that two thirds of the questions have all of their evidence within the first k results, not that two thirds of the evidence has been found. Censored questions are counted in the denominator and never removed from it. Dropping them would evaluate each retrieval strategy on whichever subset of questions it happened to succeed on, and shallower depths would be obtained simply by failing more often.

Coverage(k) is non-decreasing in k , so it can be read in the other direction as well: the shallowest depth at which a given proportion α of the questions is served is

$$k_\alpha = \min\{k : \text{Coverage}(k) \geq \alpha\}. \quad (2.9)$$

Figure 2.3 illustrates both levels. On the left, one question’s ranked list has its three gold passages at ranks 2, 5 and 7, so $K^* = 7$: reading only the top five leaves this question uncovered even though two of its three passages have appeared. On the right, the depths of a cohort of questions accumulate into the coverage curve, from which k_α is read off.

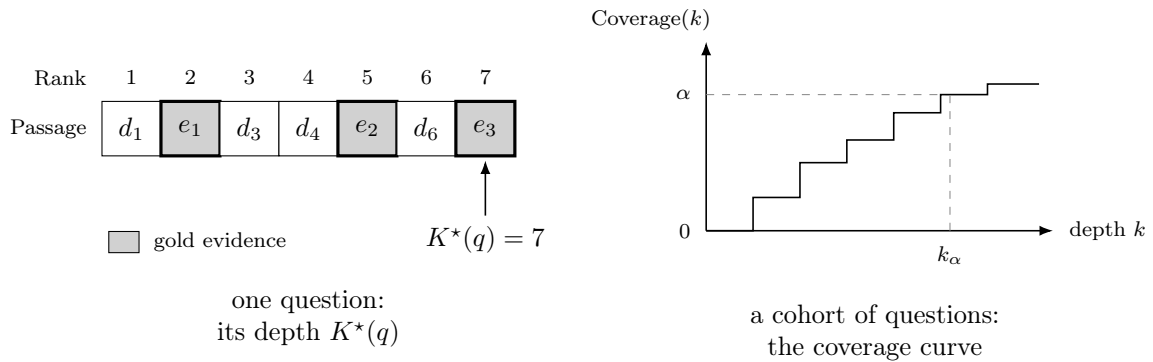


Figure 2.3: The two levels of the measurement. Left: for one question, $K^*(q)$ is the rank of its lowest-ranked gold passage; reading the top five would leave it uncovered. Right: over a set of questions those depths accumulate into Coverage(k), the proportion covered by depth k , from which the depth k_α serving a proportion α is read. The curve is schematic.

Depth here is a rank-based proxy for retrieval effort. It is neither a reasoning hop count nor a measurement of wall-clock computation, which also depends on indexing, model architecture, batching, and hardware.

Coverage and depth must therefore be read together. A useful retrieval strategy should leave few questions uncovered and should place the evidence of the rest near the top of its rankings: at equal coverage, lower depth is preferable; at equal depth, higher coverage is preferable. *Coverage* in this sense means the recovery of annotated supporting passages, and is unrelated to *conformal* or *statistical* coverage, which concerns the long-run validity of prediction sets in uncertainty quantification.

2.2 Retrieval for Multi-Hop Reasoning

Retrieval methods can be characterised along two complementary dimensions. The first concerns how relevance between a query and a candidate passage is estimated through a score $s(q, d)$: sparse, dense, and late-interaction methods provide different scoring mechanisms, while rerankers refine an initial candidate ordering. The second concerns how evidence discovery is organised. A conventional retriever performs a single search using the original query, whereas iterative and graph-based approaches use intermediate state or structured relations to guide subsequent discovery. These dimensions are not mutually exclusive. An iterative retriever may use dense scoring internally, a graph-based architecture may use lexical or dense similarity to locate and score nodes or passages, and a first-stage sparse or dense search may be followed by reranking. Of the mechanisms described below, lexical scoring and late interaction are the two used directly in the experiments of Chapter ??, where they serve to decide which supporting passage a given subquestion is responsible for. The remainder are presented because they define the vocabulary in which retrieval behaviour is described throughout this report.

2.2.1 Sparse Retrieval

Sparse retrieval represents text using high-dimensional term vectors. In a Term Frequency–Inverse Document Frequency (TF–IDF) representation, the weight of term t in document d is

$$w(t, d) = \text{tf}(t, d) \text{idf}(t), \quad \text{idf}(t) = \log \frac{N}{\text{df}(t)}, \quad (2.10)$$

where $\text{tf}(t, d)$ is the term frequency and $\text{df}(t)$ counts documents containing t . Query and document vectors can then be compared with a cosine-style similarity. TF–IDF assigns greater influence to terms that are frequent in a document but rare throughout the corpus [6].

BM25 refines lexical scoring by saturating term-frequency gains and normalising for document length [7]:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}. \quad (2.11)$$

Here $f(t, d)$ is the term frequency, k_1 controls saturation, b controls document-length normalisation, and avgdl is average document length. Sparse indices are efficient and

interpretable, and exact entity names often provide strong signals. Their principal limitation is lexical mismatch: a relevant passage can express the needed relation without sharing the query’s wording, especially when the original question hides an intermediate entity.

2.2.2 Dense Retrieval

Dense retrieval learns encoders that map text into a continuous vector space. Dense Passage Retrieval (DPR), for example, uses separate question and passage encoders [8]:

$$\mathbf{q} = f_q(q), \quad \mathbf{d} = f_d(d), \quad s(q, d) = \mathbf{q}^\top \mathbf{d}. \quad (2.12)$$

Passage vectors can be precomputed and searched with approximate nearest-neighbour methods. Because the representation is learned, semantically related text can match despite limited word overlap. This is valuable when a question and its evidence use different expressions.

The compact representation also creates a bottleneck. A single vector must compress all potentially relevant entities and relations in a passage. For a complex query that simultaneously implies several information needs, the dot product provides only a coarse aggregate match and may obscure which part of the query aligns with which part of a passage.

2.2.3 Late-Interaction Retrieval and Reranking

Contextualized Late Interaction over BERT (ColBERT) occupies a middle ground between lexical matching and single-vector dense retrieval [9]. It independently encodes contextual query tokens $\mathbf{q}_i$ and document tokens $\mathbf{d}_j$, then applies a maximum-similarity operation for every query token:

$$s_{\text{ColBERT}}(q, d) = \sum_i \max_j \mathbf{q}_i^\top \mathbf{d}_j. \quad (2.13)$$

This *MaxSim* interaction preserves fine-grained matches while allowing document representations to be precomputed, which makes it applicable wherever a decision must be made about a specific query–passage pair rather than about a whole ranking.

Retrieval systems also commonly separate inexpensive candidate generation from more expensive scoring. A fast retriever first produces a manageable candidate set; a cross-encoder or another stronger reranker jointly examines each query–passage pair and re-orders that set. Reranking can improve precision near the top but cannot recover a gold passage omitted from the candidate set. The initial retrieval stage therefore continues to constrain attainable evidence coverage.

2.2.4 Iterative and Multi-Hop Retrieval

Beyond the relevance-scoring mechanism itself, multi-hop questions raise the question of how retrieval should evolve as new information becomes available. Single-pass retrieval uses the same original question to search for every supporting passage, whereas iterative retrieval allows evidence acquired at one step to change the next query. Multi-Hop Dense Retrieval (MDR) recursively retrieves passages with a dense representation conditioned on

the question and earlier evidence, enabling later searches to incorporate newly identified bridge information [10].

Baleen addresses the rapid growth of the multi-hop search space through *condensed retrieval*: after each iteration, salient information from the retrieved material is condensed into a compact context for subsequent search. It combines this mechanism with focused late interaction and a learned ordering of retrieval steps [11].

Interleaving Retrieval with Chain-of-Thought Reasoning (IRCoT) alternates the two operations. An intermediate reasoning step specifies what information is needed next; retrieved evidence then updates the context used to produce the following step [12]. These approaches differ architecturally, but share a crucial observation: retrieval can benefit from intermediate state rather than repeatedly treating the initial question as a fixed search query.

2.2.5 Graph-Based Retrieval for Multi-Hop Reasoning

Iterative approaches organise evidence discovery through successive search steps, whereas graph-based approaches explicitly model relations among entities, passages, or concepts and use those relations to expose evidence that may be weakly connected to the original query. This is another way of organising retrieval, not a mutually exclusive alternative to sparse, dense, or late-interaction scoring: those mechanisms can still locate starting nodes or score candidate nodes and passages within a graph-based architecture. In multi-hop questions, a later passage may become relevant mainly through an entity or concept introduced by earlier evidence. Graph traversal or relevance propagation can make this connection explicit and expand beyond the passages found directly from the initial query.

HippoRAG is a representative approach. It builds a corpus-level knowledge graph, uses query–node similarity to identify starting points, and applies Personalized PageRank to propagate relevance toward connected knowledge and its source passages [13]. HippoRAG 2 retains this graph-propagation foundation while deepening the integration of passages and using an LLM more effectively during online retrieval [14]. The two systems illustrate how graph structure can help surface evidence connected through bridge entities even when that evidence is not retrieved directly from the original wording.

Graph-based retrieval therefore complements the sparse, dense, late-interaction, and iterative methods discussed above: it supplies an explicit mechanism for following associations across evidence items. What it does not do, and what none of the mechanisms in this section does, is determine which intermediate information need should be queried.

Because the depth required for complete evidence recovery varies substantially from one question to another, applying a uniformly expensive retrieval policy is neither necessary nor efficient. This observation leads to systems that estimate what an input needs and adapt their computation accordingly.

2.3 Difficulty-Aware and Adaptive Computation

Fixed pipelines allocate the same retrieval and reasoning procedure to every input, although a direct factual question and a four-step compositional question have different needs. Adaptive systems instead obtain a signal about the input or current state and translate it into an action.

Adaptive-RAG demonstrates complexity-aware routing for question answering. A classifier estimates whether a question is best handled without retrieval, with single-step retrieval, or with an iterative strategy, thereby avoiding the cost of a uniformly complex pipeline [15]. In generic form,

$$q \longrightarrow \hat{d}(q) \longrightarrow \pi_{\hat{d}}, \quad (2.14)$$

where $\hat{d}(q)$ is an estimated difficulty or complexity level and $\pi_{\hat{d}}$ is the selected computational policy. The important point is not a particular number of routing classes, but that a pre-answer estimate can govern retrieval, reasoning, or decomposition before their full costs are paid.

Adaptation can also occur after generation has begun. Forward-Looking Active Retrieval augmented generation (FLARE) anticipates the next sentence, examines confidence in the predicted content, and triggers retrieval when low-confidence tokens indicate that more information is needed. The retrieved documents are then used to regenerate the sentence [16]. Self-RAG trains a model to produce reflection signals that control retrieval on demand and support critique of both evidence and generated text [17].

Both systems make the same structural move: a quantity computed during processing is read as a signal, and that signal licenses a change of behaviour—in these cases re-retrieval, or the decision to retrieve at all.

These systems adapt on different signals. Adaptive-RAG estimates how much work a question will demand before any of that work is done; FLARE and Self-RAG estimate how reliable the content produced so far is. Both drive the same repertoire of actions—retrieve, re-retrieve, escalate—which makes the two easy to conflate, and they are not interchangeable. *Difficulty* is the amount of reasoning a question structurally requires; *uncertainty* is the reliability of what the system has produced. The two vary independently: a difficult question may be answered confidently when all the necessary evidence is available, and an easy one may remain uncertain because retrieval failed.

Existing routing approaches establish that question complexity is actionable, and they estimate it from surface characteristics with a separate classifier. That raises a representational question: could the target model itself already encode this information internally?

Before such a signal can guide an action, one must establish whether it is present in the model’s representation and can be read out reliably. Probing provides a methodology for asking that question independently of the action that may eventually consume the signal.

2.4 Probing Internal Representations of Language Models

Neural models transform an input through a sequence of hidden representations. Probing studies these representations after training, asking what information about the input or computation is accessible at a particular stage. It is an analysis methodology rather than an adaptive policy: a decoded signal may later support routing, retrieval, or another action, but the probe itself only tests whether the signal can be recovered from a frozen model.

2.4.1 Linear Probing and Representation Analysis

Let the hidden representation of question q at layer ℓ be

$$\mathbf{h}^{(\ell)}(q) \in \mathbb{R}^d. \quad (2.15)$$

Given a target property y , a probe learns a readout g and predicts

$$\hat{y} = g(\mathbf{h}^{(\ell)}(q)). \quad (2.16)$$

The language model is kept fixed while the readout is trained. Comparing readouts across layers can then reveal where a property becomes more or less accessible, without changing the representations being examined.

A linear probe intentionally restricts the readout to a simple affine map,

$$g(\mathbf{h}^{(\ell)}(q)) = \mathbf{W}\mathbf{h}^{(\ell)}(q) + \mathbf{b}, \quad (2.17)$$

followed, when needed, by a standard decision rule. Its primary purpose is not to construct the strongest possible classifier. Rather, it tests whether the property is *readily decodable* through a deliberately limited mapping. Alain and Bengio introduced this perspective by fitting linear classifiers to intermediate neural-network features and tracking how class information became linearly separable with depth [18]. In the context of LLMs, the linear representation hypothesis similarly proposes that some semantic or high-level properties correspond to directions or comparatively simple geometric structure in activation space [19]. This hypothesis motivates linear readouts without requiring a broader mechanistic account of every computation performed by the model.

Probe performance nevertheless has a limited interpretation. High accuracy shows decodability under the chosen data distribution and probe family; it does not by itself show that the model uses the decoded feature causally, or that the feature will generalise beyond the evaluation setting. Hewitt and Liang further show that an expressive probe can learn a task or exploit memorisable correlations even when the representation does not encode the intended structure in a meaningful way. They propose control tasks and selectivity as ways to put task accuracy in the context of probe capacity [20]. Probing evidence should therefore be accompanied by appropriate controls and interpreted as evidence of accessible representational structure, not as an unrestricted claim about the model’s reasoning process.

2.4.2 Latent Properties in LLM Representations

Prior work indicates that language models contain signals associated with knowledge and confidence. Kadavath et al. show that models can assess whether their proposed answers are likely to be correct and can estimate whether they know an answer, establishing a behavioural form of model self-knowledge [21]. Although this self-evaluation is not itself the same as hidden-state probing, it motivates the search for related information inside the representations that precede an answer.

Several studies recover truth-related structure directly from activations. Burns et al. identify latent knowledge without labelled examples by exploiting consistency between statements and their negations [22]. Marks and Tegmark find comparatively simple geometric structure separating true and false statements and study the transfer of truth directions across datasets [23]. Azaria and Mitchell likewise train classifiers on internal activations to distinguish truthful from false statements [24]. Together, these results

show that semantic properties such as truth can be accessible to relatively simple read-outs, while also reinforcing the need to test whether an apparent direction transfers and captures the intended property.

Question-only probing moves closer to the setting of this thesis. Moreno Cencerrado et al. extract a representation after a question has been read but before an answer is generated, and show that a linear probe can predict aspects of the eventual answer’s correctness and confidence [25]. This establishes that a question representation may contain predictive information about later success. The target considered here is different: previous work largely asks what the model knows, whether a statement is true, or whether the eventual answer is likely to be correct, whereas this thesis asks about the *compositional difficulty of the input question*—how much multi-step reasoning the question demands.

Yang et al. examine a related but distinct issue: whether LLMs internally recall and compose intermediate facts while processing a multi-hop prompt [26]. Their question concerns the execution of a latent reasoning pathway. It should not be conflated with estimating the amount of reasoning demanded by the input:

$$\underbrace{\text{Is multi-hop reasoning executed internally?}}_{\text{latent reasoning}} \neq \underbrace{\text{How much reasoning is required?}}_{\text{compositional difficulty}}. \quad (2.18)$$

The former asks what occurs during a model’s forward computation; the latter is the logically earlier question of whether the required degree of composition is already encoded in the question representation before answering.

Prior work therefore shows that internal LLM representations contain decodable information about knowledge, truthfulness, confidence, and eventual answer correctness. Considerably less attention has been given to whether the question representation itself encodes compositional difficulty before answer generation. The next section turns from this analysis methodology to question decomposition, one possible action when additional compositional reasoning is required.

2.5 Question Decomposition for Multi-Hop Reasoning

2.5.1 Why Decomposition Helps Multi-Hop Reasoning

A multi-hop question often conceals several intermediate information needs within a single surface form. Answering it may require identifying an entity, recovering a property of that entity, and only then applying a comparison or other operation. When these steps remain implicit, a model must determine the reasoning path, obtain the necessary facts, and compose them at the same time. Question decomposition instead converts the original question into simpler, explicit subproblems whose answers can be combined to obtain the final answer.

This separation is useful because knowing the answers to the component questions does not guarantee that a model can answer their composition. Press et al. describe this discrepancy as the *compositionality gap* and show that it is not necessarily removed by increasing model scale [27]. Their Self-Ask method narrows the gap by having the model generate and answer explicit follow-up questions before producing its final response. More generally, decomposition reduces a complex inference into smaller reasoning steps, makes dependencies between intermediate answers explicit, and allows an error to be associated with a particular subproblem rather than with the question as a whole.

Focused subquestions may also provide more effective retrieval queries than the original multi-hop question. Each subquestion expresses a narrower information need and can therefore be matched against evidence relevant to one step of the reasoning chain. Retrieval is not the primary objective of every decomposition method, but this property provides an important connection between explicit reasoning structure and multi-hop evidence retrieval.

2.5.2 Earlier Question-Decomposition Approaches

Question decomposition was studied before the emergence of modern instruction-tuned LLMs. These earlier approaches treated the decomposition as a learned intermediate representation between a complex input question and the single-hop models used to answer it. The intermediate subquestions made it possible to reuse existing question-answering components, but producing them typically required a task-specific architecture, training procedure, or form of decomposition supervision.

Min et al. decompose multi-hop reading-comprehension questions into simpler single-hop questions, answer the resulting questions with existing readers, and globally rescore candidate decompositions and answers. To limit the need for manually annotated decompositions, their approach casts subquestion generation as span prediction over the original question [28]. The resulting decomposition is therefore not merely an explanation appended to an answer; it is an intermediate structure on which the downstream answering process operates.

Perez et al. further reduce reliance on decomposition labels through an unsupervised one-to-many sequence-transduction model. Their system learns to map a multi-hop question to multiple single-hop subquestions, sends those subquestions to an off-the-shelf question-answering model, and recomposes the returned answers into a final prediction [29]. Together, these approaches established decomposition as a useful interface between complex questions and reusable single-hop components. At the same time, the decomposer remained a dedicated learned model designed for a particular decomposition and answering pipeline.

2.5.3 LLM-Based Decomposition

Large language models shifted question decomposition from a task-specific prediction problem toward a prompting problem. Rather than training a dedicated decomposer for each setting, a general-purpose model can be instructed—and, when useful, shown examples—to identify subproblems, solve them in an appropriate order, and combine their results. Self-Ask is an early example of this pattern: the prompt elicits follow-up questions whenever an intermediate fact is needed [27].

Successive Prompting interleaves decomposition and answering. At each step, the model generates a simpler question, answers it, and uses the accumulated context to determine the next question until the original problem is resolved [30]. Least-to-Most Prompting separates these phases more clearly: it first decomposes a complex problem into a sequence of simpler subproblems and then solves them from least to most complex, using earlier solutions to support later ones [31]. Decomposed Prompting adopts a modular formulation in which a controller prompt breaks a task into subtasks and delegates them to specialised prompts, models, or symbolic modules [32].

Although these methods differ in when decomposition occurs and how subproblems are

solved, they replace a fixed learned decomposer with behaviour elicited largely through prompting. Decomposition consequently becomes a configurable reasoning component: instructions, demonstrations, output formats, and available modules can change both the number and the form of the generated subquestions. This flexibility makes decomposition easier to adapt across tasks, while also making the quality of the prompt and of the generated intermediate structure consequential for all subsequent reasoning and retrieval.

2.6 Research Gap and Thesis Positioning

The preceding literature connects complex questions to adaptive computation, probing, and decomposition, but leaves two questions that structure the experimental chapters of this thesis.

2.6.1 Anticipating Compositional Difficulty

Adaptive computation benefits from an estimate of query difficulty before an expensive policy is selected. At the same time, prior probing work shows that knowledge, truth, confidence, and eventual answer correctness can be decoded from internal representations. Whether those representations also encode the compositional difficulty of the question itself remains unresolved and is the first research question of this thesis.

Does the LLM already encode how much compositional reasoning a question requires?

Chapter 3 investigates this question while keeping compositional difficulty conceptually distinct from expected answer correctness.

2.6.2 Optimising Decomposition for Retrieval Efficiency

Question decomposition exposes intermediate information needs, and iterative retrieval demonstrates that such intermediate state can improve search. The remaining issue is whether an arbitrary generated decomposition can be judged and improved specifically as a set of retrieval queries.

Can a decomposition preserve evidence coverage while reducing the maximum depth required to retrieve that evidence?

Answering this requires three capabilities: determining which evidence each subquestion is responsible for, evaluating decompositions jointly through coverage and depth, and adapting the decomposition-generation strategy according to those metrics. Chapter ?? develops the evidence-aware evaluation and attribution required by the first two. Chapter ?? then uses the resulting signal to optimise the decomposition prompt.

Conclusion

This chapter presented the background necessary for the experimental work in the remainder of the thesis. Multi-hop question answering requires both reasoning across several pieces of evidence and retrieval mechanisms capable of exposing that evidence

efficiently. Existing work approaches these demands through improved retrieval, adaptive computation, and question decomposition, while probing provides a means of identifying internal signals that may support such adaptation.

Two questions remain central, and they approach the same difficulty from opposite sides: one asks whether the amount of reasoning a question requires is already visible inside the model, the other asks how the retrieval work that this amount implies can be reduced outside it. The first is whether compositional difficulty is encoded in the question representation before answering, which is investigated in Chapter 3. The second is whether decomposition can be evaluated and optimised to preserve evidence coverage while reducing retrieval depth, which motivates Chapters ?? and ??.

Chapter 3

Probing Compositional Question Difficulty in Language Model Representations

Introduction

The preceding chapter established the importance of anticipating how much compositional reasoning a question may require before committing to a fixed retrieval or reasoning strategy. This chapter examines whether such difficulty is already reflected in the internal representations of a language model while it processes the question, before any answer is generated.¹

Hidden states are extracted across the layers of three open instruction-tuned models and read with simple linear probes on three multi-hop benchmarks whose notions of difficulty differ in kind; every positive result is then examined through a series of controls.

3.1 Research Question and Problem Formulation

Let

$$\mathcal{D} = \{(q_i, y_i)\}_{i=1}^N \quad (3.1)$$

be a collection of questions, where q_i is a question and y_i is its difficulty label. Depending on the benchmark, y_i may be a binary easy–hard split, a human-assigned difficulty level, or a structural hop count. These labels provide operational proxies for compositional difficulty: they make the question measurable, but they do not constitute a universal definition of difficulty.

For a frozen language model M , processing question q_i produces a representation

$$\mathbf{r}_i^{(\ell)} \in \mathbb{R}^d \quad (3.2)$$

at layer ℓ , where d is the hidden dimension. The principal research question is whether a simple function of $\mathbf{r}_i^{(\ell)}$ can recover y_i , and how that recoverability changes across model depth. More precisely, the experiments ask whether there exists a linear score

$$s_i^{(\ell)} = \mathbf{w}^{(\ell)\top} \mathbf{r}_i^{(\ell)} \quad (3.3)$$

¹The study reported in this chapter was carried out during the internship and has been accepted for publication [5]. The present author is its first author and was responsible for the design and implementation of the probing pipeline, its validation controls, and the analysis of the results. The chapter develops the study in extended form.

that orders questions according to their benchmark-defined difficulty on examples not used to fit the direction $\mathbf{w}^{(\ell)}$. How that direction is obtained is the subject of Section 3.2.2.

Decodability is the weakest of the four properties established here. A representation that genuinely encodes compositional structure should also exceed what the same procedure yields when the labels are shuffled, survive the removal of the question’s surface statistics, order the difficulty levels rather than merely separate them, and hold across benchmarks that define difficulty differently. The four results follow in that order:

1. difficulty is decodable from the question representation across model depth;
2. the observed performance exceeds a shuffled-label null and is not exhausted by six selected surface features;
3. the decoded direction orders an unseen intermediate difficulty class between easy and hard extremes; and
4. the direction, after controlling for surface features, transfers across benchmarks within the representation space of the same model.

Figure 3.1 summarises this logic. The language model is never trained or fine-tuned and no answer is generated: only the representations formed during the question’s forward pass are studied.

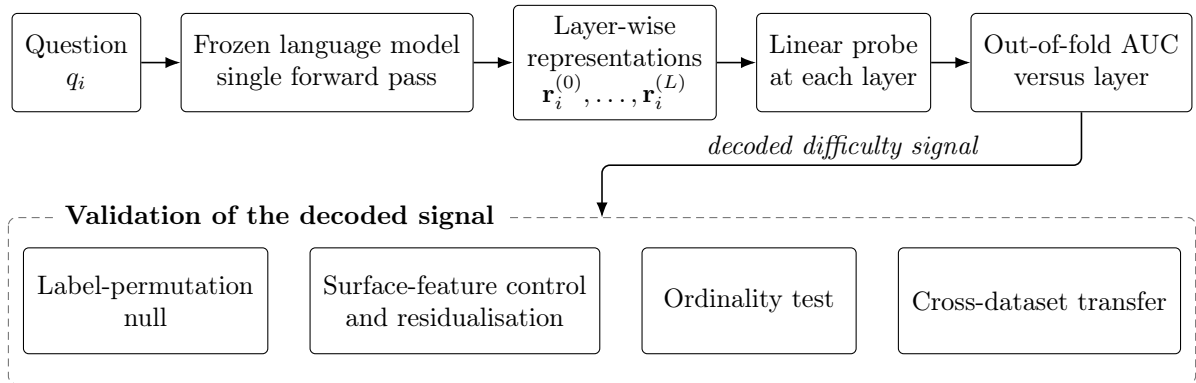


Figure 3.1: Overview of the compositional-difficulty probing protocol. A frozen language model processes each question once, and a linear probe is evaluated independently at each layer using out-of-fold predictions. The decoded signal is subsequently examined through permutation, surface-feature, ordinality, and cross-dataset transfer analyses.

3.2 Methodology

3.2.1 Hidden-State Extraction and Question Representation

Each question is tokenised as raw text, without a chat template, and passed through a frozen causal language model once. Hidden states are read from all $L + 1$ positions in depth: layer 0 is the token-embedding output, while layers $1, \dots, L$ are the outputs of the transformer blocks. Extraction is forward-pass only; the model does not generate an answer.

Because questions contain different numbers of tokens, each layer’s token-level states must be reduced to one fixed-dimensional question vector. Let $\mathbf{h}_{it}^{(\ell)}$ be the hidden state of token t for question i at layer ℓ , and let $m_{it} \in \{0, 1\}$ denote its attention mask. The reported experiments use mask-aware last-token pooling:

$$t_i^* = \max\{t : m_{it} = 1\}, \quad \mathbf{r}_i^{(\ell)} = \mathbf{h}_{it_i^*}^{(\ell)}. \quad (3.4)$$

The maximum in Equation (3.4) identifies the final non-padding token, ensuring that padding is never selected. Last-token pooling is used because the model is causal: the final non-padding position is the only one whose representation has attended to the whole question. Mean pooling over the question’s tokens is an equally defensible choice and is not evaluated here. For each question, the complete extracted representation is therefore

$$\mathbf{R}_i = \begin{bmatrix} \mathbf{r}_i^{(0)} \\ \mathbf{r}_i^{(1)} \\ \vdots \\ \mathbf{r}_i^{(L)} \end{bmatrix} \in \mathbb{R}^{(L+1) \times d}. \quad (3.5)$$

Keeping one vector per layer makes it possible to determine not only whether difficulty is decodable, but also where in the network the signal emerges and where it is strongest.

3.2.2 Linear Probe and Layer-Wise Evaluation

The readout is deliberately the simplest one that can still order questions. At a given layer it averages the representations of the difficult questions, averages those of the easy questions, and takes the difference between the two averages; a new question is then scored by how far along that direction it falls. Formally, this is a regularisation-free diagonal Linear Discriminant Analysis (LDA) probe. Its simplicity is the point: the activation dimension d is several thousand while the number of questions N is at most 900, and a classifier with more freedom could fit the labels in that regime without reading anything from the representation.

All preprocessing is fitted within the training part of each fold. For feature j , the standardised activation is

$$\hat{r}_{ij}^{(\ell)} = \frac{r_{ij}^{(\ell)} - \mu_j^{(\ell)}}{\sigma_j^{(\ell)}}, \quad (3.6)$$

where the mean $\mu_j^{(\ell)}$ and standard deviation $\sigma_j^{(\ell)}$ are estimated only from the training fold. For a binary contrast, let $\bar{\mathbf{r}}_+$ and $\bar{\mathbf{r}}_-$ be the two class means in standardised space. The probe direction and score are

$$\mathbf{w} = \bar{\mathbf{r}}_+ - \bar{\mathbf{r}}_-, \quad s_i = \mathbf{w}^\top \hat{\mathbf{r}}_i. \quad (3.7)$$

The direction is therefore fixed entirely by the two class means: the probe has no weights to tune and no other free parameters.

The same construction covers the benchmarks that have three difficulty levels. It is applied once per level: that level takes the role of the difficult class and the other two together take the role of the easy one, which gives one direction for each level.

Evaluation uses five-fold cross-validation. Each question receives a score from a probe that was not trained on that question, and the pooled out-of-fold scores are used to

compute the Area Under the Receiver Operating Characteristic Curve (AUC). AUC has a direct reading in this setting: it is the probability that a randomly chosen difficult question receives a higher score than a randomly chosen easy one, so 0.5 corresponds to chance and 1.0 to a perfect ordering. The procedure is repeated independently at every layer to form an AUC-versus-layer curve. Where there are three levels, each of those directions is scored separately and the three AUCs are averaged; this average is what the macro one-versus-rest AUC means in the tables that follow. The binary 2Wiki contrast has a single direction and a single AUC. AUC is the primary measure because it evaluates ranking without requiring a deployment threshold.

3.2.3 Label-Permutation Null

A probe is fitted at every one of several dozen layers, using several thousand features each time. A high value somewhere in that grid could therefore appear even when questions and labels have no relation at all. The label-permutation control measures how large such a value can be when that relation is removed by construction. The complete probing procedure is repeated for $P = 200$ random shuffles of the labels, yielding a null distribution at each layer.

The empirical AUC curve is compared with the null’s 5th–95th percentile band. If the real curve lies well above that band, its performance cannot be attributed to the capacity of this probing procedure under randomly associated labels. This is a statistical control for decodability; it does not, by itself, rule out meaningful but superficial correlates of the true labels.

3.2.4 Surface-Feature Control and Residualisation

Benchmark difficulty may be correlated with visible properties of the question. Longer questions, for instance, may tend to contain more relations. The study collects six inexpensive surface statistics in ϕ_i : character length, word count, capitalised-word count as a named-entity proxy, digit-token count, punctuation count, and tokenised length. A surface-only probe establishes how predictive these statistics are on their own.

The residual test asks what is left in the hidden state once those cues have been taken out of it. For every activation feature, the part of its value that can be predicted from ϕ_i is estimated and subtracted, leaving only the part the six statistics cannot account for. The prediction is fitted inside each training fold, giving coefficients $\mathbf{B}^{(\ell)}$; both training and held-out examples are then transformed as

$$\tilde{\mathbf{r}}_i^{(\ell)} = \mathbf{r}_i^{(\ell)} - \mathbf{B}^{(\ell)\top} \phi_i. \quad (3.8)$$

The linear probe is then fitted to these residualised representations. Because the regression coefficients and all subsequent preprocessing are estimated inside the training fold, the held-out labels and activations do not influence the transformation used to score them.

The comparison has three parts: the raw activation probe measures total decodability, the surface-only probe measures the selected visible cues, and the residual probe measures the component not linearly explained by those cues. A positive residual result supports a component beyond the six selected features; it does not prove complete independence from every possible aspect of surface form.

3.2.5 Ordinality Test

For benchmarks with at least three ordered classes, class separation need not imply a graded representation. A classifier might distinguish unrelated templates for each label without arranging them along one difficulty axis. The ordinality test therefore fits $\mathbf{w}$ using only the lowest and highest difficulty classes, then projects the unseen middle class onto that direction.

Let $\bar{s}_{\text{low}}$, $\bar{s}_{\text{mid}}$, and $\bar{s}_{\text{high}}$ be the out-of-fold mean projection scores for the three classes. Their interpolation fraction is

$$\rho^{(\ell)} = \frac{\bar{s}_{\text{mid}}^{(\ell)} - \bar{s}_{\text{low}}^{(\ell)}}{\bar{s}_{\text{high}}^{(\ell)} - \bar{s}_{\text{low}}^{(\ell)}}. \quad (3.9)$$

A value near 0 places the middle class close to the easy extreme, a value near 1 places it close to the hard extreme, and $0 < \rho^{(\ell)} < 1$ places it between the two. The central point is that the middle class is never used to define the direction; interpolation therefore tests an ordering that the probe was not trained to impose.

For MuSiQue, the extremes are 2-hop and 4-hop questions and the held-out middle class is 3-hop. For HotpotQA, the easy and hard labels define the axis and the medium class tests interpolation. The binary 2Wiki split cannot support this three-level test.

3.2.6 Cross-Dataset Transfer

The final validation asks whether a decoded direction captures a property that is shared across benchmarks rather than their individual topics or templates. A direction fitted on the extreme classes of source benchmark A is applied to the extreme classes of target benchmark B , within the activation space of a fixed language model. Transfer is summarised by the retention ratio

$$R_{A \rightarrow B}^{(\ell)} = \frac{\text{AUC}_{A \rightarrow B}^{(\ell)} - \frac{1}{2}}{\text{AUC}_B^{(\ell)} - \frac{1}{2}}, \quad (3.10)$$

where $\text{AUC}_{A \rightarrow B}$ is obtained by training on A and testing on B , and AUC_B is the target’s own within-benchmark, cross-validated extreme-class score. Chance is 0.5, so subtracting it from both terms compares the two scores by how far each rises above chance rather than by their raw values. A retention near 1 means the transferred direction recovers what a probe trained on the target benchmark itself achieves; $R = 0$ corresponds to chance transfer and $R < 0$ to below-chance transfer.

The reported principal analysis uses a fused leave-one-benchmark-out variant: for each target, the extreme classes of the other two benchmarks are pooled to fit the axis, which is then evaluated on the held-out benchmark’s extreme classes. AUC and retention are averaged over the informative layer band in which the target benchmark’s own extreme-class probe performs most strongly. Selecting this band from the target’s within-benchmark curve, rather than from the transfer curve, prevents the reported transfer from being chosen at its own maximum.

Transfer here is *cross-benchmark*, not cross-model. Every transferred axis is trained and tested on representations produced by the same language model. Whether an axis learned in one model can be applied directly to another model is outside the scope of the experiment.

3.3 Experimental Setup

3.3.1 Language Models

The experiments cover three open instruction-tuned language models: Qwen2.5-32B, Mistral-24B, and Gemma-27B. They have 65, 41, and 47 hidden-state positions respectively, counting the embedding output as position 0. Each model is probed only on the activations that it produced itself. This design supports replication of the finding across three model families, but it does not evaluate transfer of a probe between models.

3.3.2 Benchmarks and Difficulty Labels

Three English, Wikipedia-based multi-hop question-answering benchmarks provide deliberately different difficulty definitions:

- **2WikiMultiHopQA** [3] supplies a binary contrast derived from evidence-chain length. Questions with at most two evidence items form the easy class, and questions with at least four form the hard class.
- **HotpotQA** [2] provides human-annotated easy, medium, and hard difficulty levels.
- **MuSiQue** [4] supplies a structural composition label of 2-hop, 3-hop, or 4-hop.

The analysis samples 300 questions per class, producing 600 questions for the binary 2Wiki task and 900 each for HotpotQA and MuSiQue. The class-balanced design prevents class prevalence from dominating the evaluation. More importantly, the variation in label type makes agreement across benchmarks stronger evidence than repeated evaluation on one construction scheme. It also requires a conservative interpretation: evidence-chain length, human judgement, and hop count are related proxies, not interchangeable definitions of an abstract latent quantity.

3.3.3 Evaluation Configuration

The procedure of Section 3.2 is applied unchanged to all nine model–benchmark pairs. Two reporting conventions hold throughout. Confidence intervals are the half-width of a 95% bootstrap interval over questions, from 2,000 resamples; the reference quoted from the label-permutation null is its 95th percentile over the 200 shuffles.

Unless stated otherwise, the figures use Qwen2.5-32B on MuSiQue as a running example, while the tables report all nine model–benchmark combinations. These choices keep the layer-wise plots readable without reducing the breadth of the numerical comparison.

3.4 Results

3.4.1 Layer-Wise Decodability of Compositional Difficulty

Table 3.1 shows that compositional difficulty is linearly decodable above chance for every model–benchmark pair. Structural labels are especially separable: the 2Wiki binary contrast reaches an AUC from 0.972 to 0.987, and MuSiQue’s three-class hop count reaches

Table 3.1: Difficulty is linearly decodable from question representations beyond the selected surface features. N is the sample size, C the number of classes, and Layer the best-performing hidden-state position, with embeddings at 0. Probe AUC is macro one-versus-rest AUC, except for binary 2Wiki. “Surf. resid.” is the peak after fold-local surface residualisation; Null95 is the 95th percentile of the 200-run permutation null. The $\pm$ value is the half-width of a 95% bootstrap interval over questions using 2,000 resamples.

Model	Benchmark	N	C	Layer	Probe AUC	Surf. only	Surf. resid. / Null95
Qwen2.5-32B	2Wiki	600	2	64	0.987 ± 0.006	0.907	0.838 ± 0.034 / 0.55
	HotpotQA	900	3	44	0.716 ± 0.024	0.661	0.627 ± 0.027 / 0.53
	MuSiQue	900	3	24	0.896 ± 0.013	0.779	0.774 ± 0.022 / 0.53
Mistral-24B	2Wiki	600	2	21	0.972 ± 0.010	0.911	0.829 ± 0.035 / 0.55
	HotpotQA	900	3	18	0.694 ± 0.025	0.660	0.604 ± 0.027 / 0.53
	MuSiQue	900	3	12	0.874 ± 0.015	0.776	0.753 ± 0.023 / 0.53
Gemma-27B	2Wiki	600	2	44	0.978 ± 0.010	0.909	0.833 ± 0.035 / 0.54
	HotpotQA	900	3	23	0.709 ± 0.024	0.661	0.606 ± 0.027 / 0.53
	MuSiQue	900	3	24	0.880 ± 0.015	0.780	0.766 ± 0.023 / 0.53

a macro AUC from 0.874 to 0.896. HotpotQA’s human-labelled levels are less separable, with AUC from 0.694 to 0.716, consistent with a noisier and less purely structural proxy.

The layer profile is as important as the peak. Figure 3.2 shows that Qwen2.5-32B is at chance at the embedding layer on MuSiQue, after which decoding performance rises through the transformer and peaks at layer 24. The same emergence profile occurs in all nine settings: the signal is absent from the token embeddings and becomes decodable as the question is processed. Where it peaks, however, tracks the benchmark’s notion of difficulty more closely than it tracks the model. The structural hop count of MuSiQue is decoded earliest, at roughly a third to a half of network depth: layer 24 of 65 for Qwen2.5-32B, layer 12 of 41 for Mistral-24B, and layer 24 of 47 for Gemma-27B. HotpotQA’s human annotation peaks nearer the middle, and the 2Wiki evidence-chain contrast peaks latest, at or close to the final layer in two of the three models. What replicates across settings is therefore the emergence of the signal with depth, not one preferred layer.

This trajectory supports a representational interpretation stronger than a single high peak: the final token’s contextual representation acquires difficulty-related structure across depth. It remains a statement about what is linearly readable, however, not proof that subsequent model computation uses that structure.

3.4.2 Robustness Beyond Surface Features

The first control verifies that the decoding curve is not reproduced when difficulty labels are detached from their questions. Across settings, the 95th-percentile permutation reference remains around 0.53–0.55, far below the observed peaks in Table 3.1. Figure 3.3 shows the complete 200-shuffle null band for the Qwen2.5-32B–MuSiQue example. The empirical curve rises well above the band over a broad span of layers rather than at one isolated position.

Surface form nevertheless explains a meaningful part of performance. A probe using only the six surface statistics reaches 0.907–0.911 on 2Wiki and 0.776–0.780 on MuSiQue, so the raw result cannot be described as independent of visible question properties. The

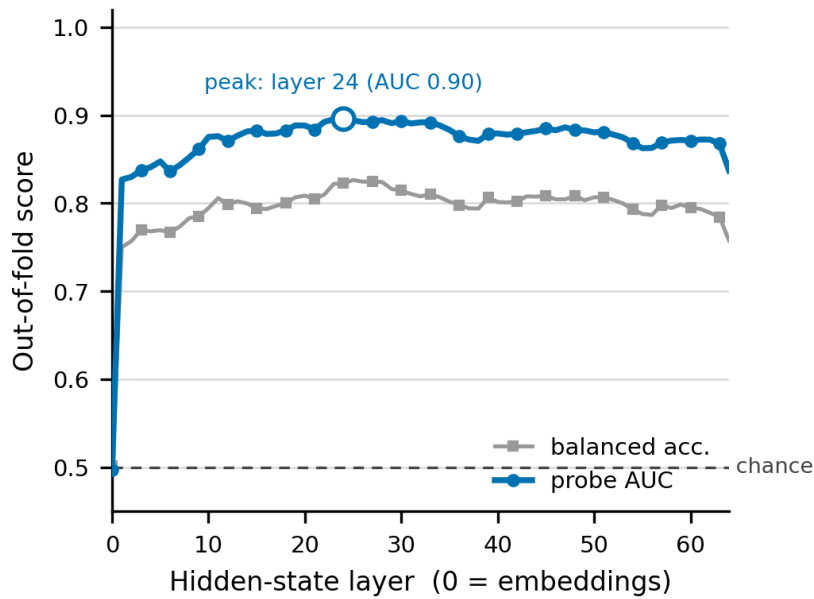


Figure 3.2: Difficulty becomes linearly decodable as the question is processed. The figure reports out-of-fold macro one-versus-rest AUC and balanced accuracy by hidden-state position for a diagonal-LDA probe on Qwen2.5-32B and MuSiQue. Performance begins near chance at the embedding representation and peaks in the intermediate layers. Adapted from [5].

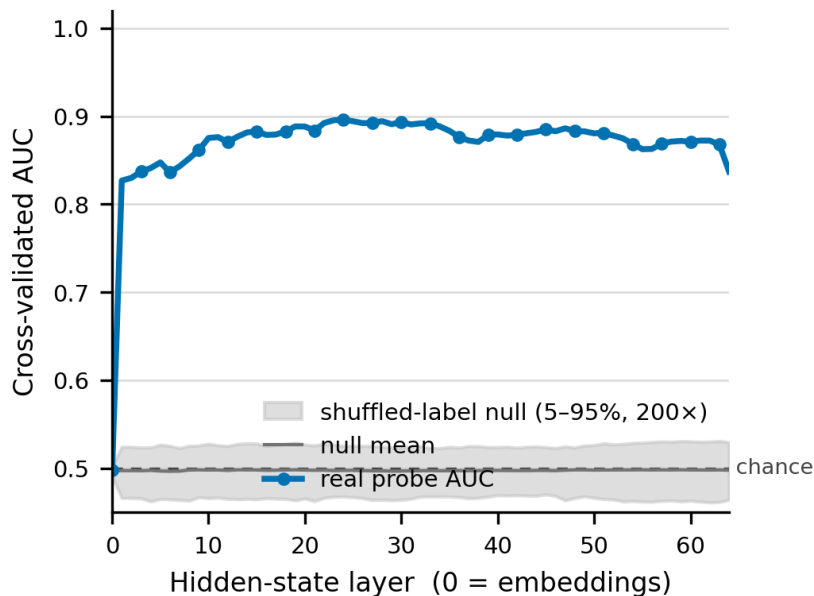


Figure 3.3: The real probe curve lies well above the 5th–95th percentile band obtained from 200 shuffled-label runs for Qwen2.5-32B on MuSiQue. The control shows that the observed layer-wise decodability is not reproduced by the probing pipeline when question–label correspondence is destroyed. Adapted from [5].

residual analysis asks the narrower and more defensible question: does a decodable component remain after removing the linear contribution of those selected properties?

The answer is positive for all model–benchmark pairs. Residual peak AUC is 0.829–0.838 on 2Wiki, 0.753–0.774 on MuSiQue, and 0.604–0.627 on HotpotQA, all above the corresponding null reference. Figure 3.4 shows that the residual curve for Qwen2.5-32B on MuSiQue remains elevated across depth even though it is lower than the raw curve.

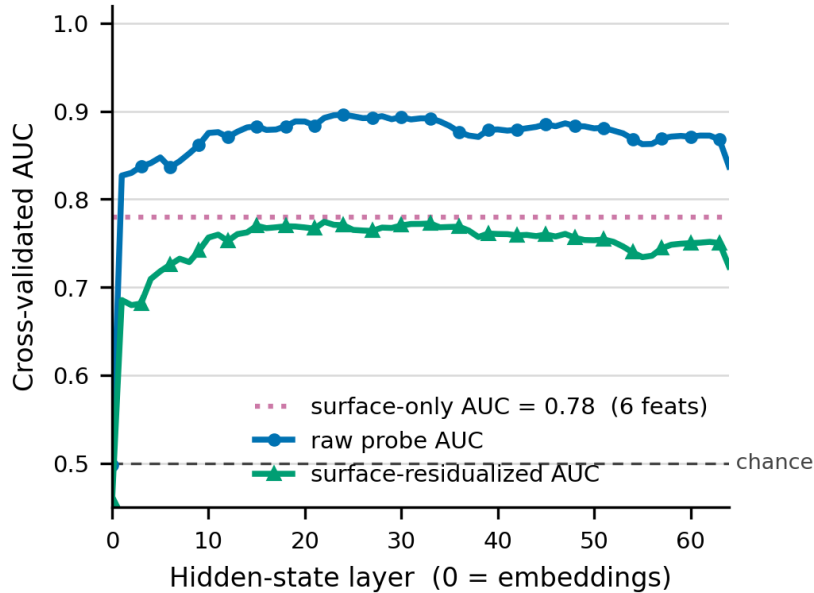


Figure 3.4: Raw and surface-residualised probe AUC across depth, with the surface-only score as a reference, for Qwen2.5-32B on MuSiQue. Removing six selected surface features reduces performance but leaves a substantial above-null signal. This supports a component beyond those features, not complete independence from all surface form. Adapted from [5].

Together, the two controls delimit the claim. The signal is not a shuffled-label artefact, and the six measured surface statistics do not exhaust it. At the same time, the strength of surface-only prediction shows that compositional difficulty is partly expressed in the wording and structure of the input.

3.4.3 Ordinal Structure of the Difficulty Direction

The extreme-class probes separate low from high difficulty almost perfectly on MuSiQue, reaching AUC values up to 0.99. More importantly, the unseen 3-hop class falls between the 2-hop and 4-hop extremes. Its interpolation fraction is 0.58 for Qwen2.5-32B, 0.56 for Mistral-24B, and 0.47 for Gemma-27B. These values cluster around the midpoint despite the fact that 3-hop questions never define the direction.

Figure 3.5 displays both the strength of the extreme-class contrast and the position of the held-out class. HotpotQA exhibits the same ordering, although its medium class lies closer to the hard extreme, with interpolation fractions from 0.93 to 0.96. The result therefore does not claim equal spacing among labels. It shows that the representation arranges the middle class in the expected order along a direction learned only from the endpoints, which is evidence for a graded difficulty axis rather than three disconnected class templates.

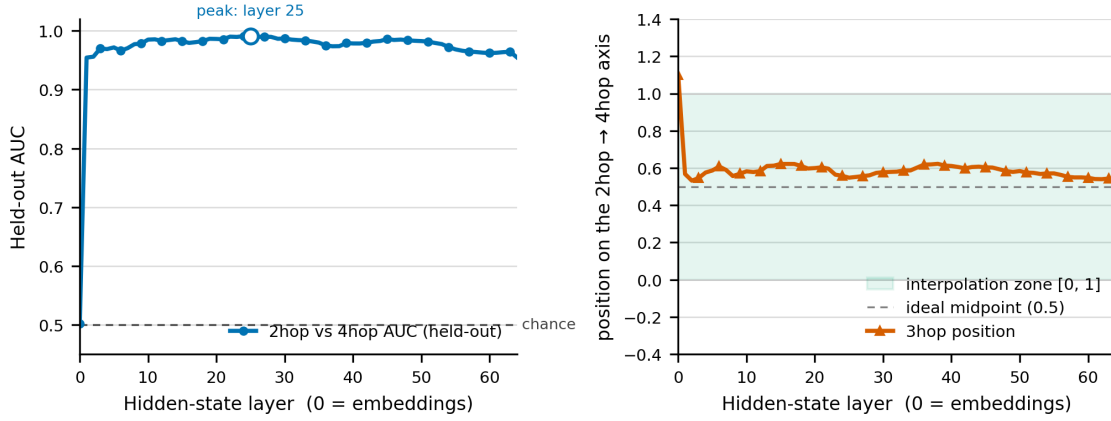


Figure 3.5: Left: out-of-fold AUC for the 2-hop versus 4-hop contrast by hidden-state position. Right: the interpolation fraction of the held-out 3-hop class. Across informative depths, the intermediate class lies between the two extremes it was not used to fit. Results are for Qwen2.5-32B on MuSiQue. The peak differs from the three-class peak because this test uses a binary extreme-class contrast. Adapted from [5].

3.4.4 Cross-Dataset Transfer of the Residualised Difficulty Direction

Table 3.2 presents the fused leave-one-benchmark-out analysis. The raw axis transfers unevenly. It retains 0.54–0.71 of above-chance within-benchmark performance on 2Wiki and only 0.25–0.39 on MuSiQue; on HotpotQA, raw transfer is below chance, producing negative retention from -0.48 to -0.42 . This failure is consistent with the raw axis mixing compositional structure with dataset-specific surface cues. Because retention is a ratio to each benchmark’s own within-benchmark reference, and that reference is weak for HotpotQA, values above 1 occur; their interpretation is given after the table.

After surface residualisation, transfer becomes strong in all nine cases. Retention rises to 0.80–0.98 for 2Wiki, 1.02–1.29 for MuSiQue, and 1.95–2.44 for HotpotQA. Figure 3.6 illustrates the Qwen2.5-32B transfer-to-MuSiQue case: the axis is fitted on the pooled extreme classes of 2Wiki and HotpotQA, then evaluated on MuSiQue.

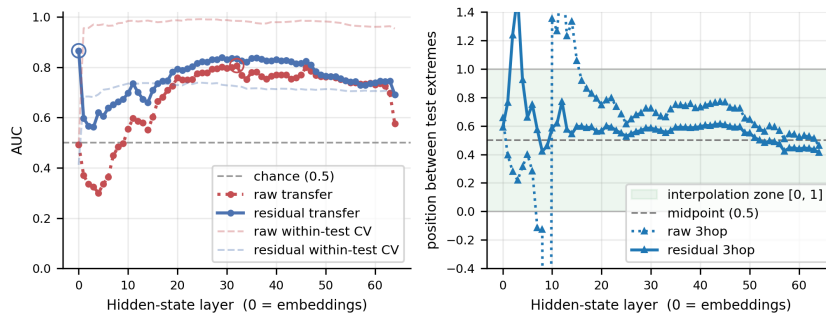


Figure 3.6: Transfer to MuSiQue in the representation space of Qwen2.5-32B, with the difficulty direction fitted on pooled extreme classes from 2Wiki and HotpotQA. The raw direction transfers weakly, whereas surface residualisation yields strong transfer across the informative layer band. This is cross-benchmark transfer within one model, not transfer between model families. Adapted from [5].

Table 3.2: Fused cross-benchmark transfer within each model. For each test benchmark, the axis is fitted on pooled extreme classes from the other two benchmarks. AUC is the mean binary extreme-class score over informative layers, and $R = (\text{AUC} - 0.5)/(\text{AUC}_{\text{within}} - 0.5)$ is retention relative to the test benchmark’s own within-benchmark score. Raw uses activations directly; Resid. first removes the six surface features. These binary extreme-class AUCs are not directly comparable to the three-class macro peaks in Table 3.1.

Model	Test benchmark	Raw		Residualised	
		AUC	R	AUC	R
Qwen2.5-32B	2Wiki	0.823	0.71	0.796	0.94
	HotpotQA	0.384	-0.43	0.774	1.95
	MuSiQue	0.685	0.39	0.769	1.22
Mistral-24B	2Wiki	0.774	0.61	0.779	0.98
	HotpotQA	0.379	-0.48	0.764	2.21
	MuSiQue	0.627	0.27	0.720	1.02
Gemma-27B	2Wiki	0.744	0.54	0.724	0.80
	HotpotQA	0.398	-0.42	0.756	2.44
	MuSiQue	0.610	0.25	0.768	1.29

A retention greater than 1 does not mean that more than 100% of difficulty has been recovered. Retention normalises the transferred AUC above chance by a within-benchmark reference. HotpotQA has a comparatively weak residual within-benchmark signal, so a cleaner axis pooled from two other benchmarks can exceed that reference and produce a large ratio. The central finding is the consistent direction of the residual transfer, including between a human difficulty annotation and structural hop-based labels. Within each model’s representation space, the component beyond the selected surface features is therefore substantially more benchmark-general than the raw axis.

3.5 Discussion and Limitations

3.5.1 Interpretation and Implications for Adaptive Computation

The experiments support a coherent representational account. Difficulty is not linearly available in the embedding-layer representation of the last input token, but becomes decodable as contextual processing proceeds; where it is strongest depends on the benchmark’s notion of difficulty, from roughly a third of network depth for hop counts to the final layers for the evidence-chain contrast. The decoded direction also has ordinal structure: it places an unseen middle class between extremes. Finally, surface residualisation weakens the within-benchmark signal yet makes its cross-benchmark behaviour markedly more consistent. Taken together, these results suggest that the model forms a graded representation related to how much compositional reasoning a question requires.

The comparison with the surface-only probe deserves a direct statement, since those six statistics are themselves strong predictors. Read as a predictor, the probe is the better of the two in all nine settings, by between 0.03 and 0.12 AUC. Read as a signal, the gap

is wider than that margin suggests. The raw axis, which still carries surface information, transfers poorly between benchmarks and inverts on HotpotQA, whereas the residualised axis transfers in every case. Surface statistics are therefore a reasonable predictor within the benchmark for which they were chosen and an unreliable one outside it. The residualised probe is not proposed as a better predictor—it is deliberately handicapped—but as the measurement showing that the part of the signal which survives the controls is also the part that generalises.

The implication for the adaptive computation of Chapter 2 is concrete, and this chapter stops short of it. A score read from a forward pass that a system already performs could gate whether a question is decomposed at all, or how many subquestions are generated for it; Chapters ?? and ?? build that decomposition machinery but apply it to every question rather than under such a gate. Nothing here calibrates a threshold, routes a query, or measures whether using the score improves efficiency or answer quality. Nor does a probe show that the model itself consults this direction when allocating computation: decodability is not use, and joining the two is a continuation this report does not attempt.

3.5.2 Limitations and Future Directions

Several limitations bound the conclusions.

Difficulty is partly surface form. The surface-only probe reaches 0.91 on 2Wiki, so visible features carry real information about the benchmark labels. Residualisation removes six specified linear surface effects, not every lexical, syntactic, or semantic correlate. One it does not touch at all: 2WikiMultiHopQA is generated from templates, so its easy and hard classes may differ in template as well as in evidence-chain length, and 2Wiki is where the surface-only probe is strongest. A subsample matched on template or length would isolate this factor.

Benchmark labels are heterogeneous proxies. Hop count approximates the number of compositional steps, evidence-chain length defines a different structural split, and HotpotQA difficulty is a human annotation. HotpotQA is both the least decodable label and the source of the largest residual retention ratios because its within-benchmark reference is weak. The results decode these operational labels rather than difficulty in the abstract.

The evidence is correlational. No activation intervention or ablation tests whether the direction has a causal role; establishing one would require modifying the decoded component and measuring the resulting change in model behaviour.

The empirical scope is narrow. The study covers three open instruction-tuned models in the 24–32 billion parameter range, English Wikipedia-based multi-hop question answering, last-token pooling, and one regularisation-free linear probe family. Generalisation beyond that scope—to other model sizes, base models, architectures, languages, domains, task types, token positions, pooling choices, or nonlinear probes—is untested.

Each of these bounds indicates its own continuation: causal intervention on the decoded direction, its relation to whether a model actually executes the required reasoning, further model families and scales, and a calibrated in-model difficulty estimate for adaptive computation.

Conclusion

This chapter showed that benchmark-defined compositional question difficulty is linearly decodable from the question representations of three language models. The signal emerges across depth, survives a permutation null and partial control for surface form, behaves ordinally, and transfers across benchmarks more consistently after surface residualisation. These findings establish a readable representational signal, not its causal use or an implemented router.

This chapter answers the first of the two questions raised in Chapter 2, at the level of the model’s internal representation. The chapters that follow address the second at a complementary level: instead of asking whether the difficulty of a question can be read before retrieval begins, they ask how the retrieval effort that a difficult question implies can be reduced once it does. The two studies are deliberately independent—no quantity computed here is consumed by what follows—and are joined by the problem they share rather than by a shared pipeline.

Chapter 4

Evidence-Aware Evaluation of Question Decomposition

Introduction

Chapter 3 examined how compositional difficulty is represented before a model answers a question. This chapter studies the retrieval process that follows a decision to decompose. Its objective is to measure whether the resulting subquestions retrieve the evidence needed for the original question, and how deeply each relevant retrieval list must be searched.

A decomposition changes several parts of the computation at once. It creates additional retrieval queries, assigns information needs to those queries and passes intermediate answers between them. Final-answer accuracy combines these effects. Passage recall measures evidence availability but leaves the assignment of that evidence to individual subquestions unspecified. Evaluating the decomposition therefore requires three separate quantities: complete evidence availability, evidence ownership and successful answer execution.

The chapter develops an evaluation based on a matrix of evidence ranks and a learned ownership mask. The mask identifies which supporting passages each subquestion is intended to retrieve. A maximum over the selected cells measures the depth required to satisfy those assignments. A separate union measurement records whether the evidence was retrieved anywhere in the execution. The ownership model is evaluated on reference-style decompositions whose assignments are known. Retrieval and answer results are then compared on a 100-question MuSiQue panel. Finally, fixed-list budget replay distinguishes per-query depth from total passage expenditure. These measurements provide the objective and diagnostic information used for prompt optimisation in Chapter 5.

4.1 Problem Formulation

4.1.1 A Decomposition as an Executable Sequence

Let q be a question with reference answer y and annotated supporting passages $E(q) = \{e_1, \dots, e_h\}$. MuSiQue supplies both supporting passages and a reference decomposition into connected single-hop questions [4]. A generated decomposition is an ordered sequence $U(q) = (u_1, \dots, u_A)$. An item u_a may contain a reference to the answer of an earlier item. The execution substitutes the predicted earlier answer, retrieves passages for the resulting

query $\tilde{u}_a$ and produces an intermediate answer $\hat{y}_a$:

$$\tilde{u}_a = \text{substitute}(u_a, \hat{y}_1, \dots, \hat{y}_{a-1}), \quad \hat{y}_a = \text{reader}(\tilde{u}_a, R_a[1 : k]). \quad (4.1)$$

Here $R_a = (d_{a1}, d_{a2}, \dots)$ is the ranked passage list returned for $\tilde{u}_a$, and k is the number of passages supplied to the reader. Backward references make the execution sequential even when the decomposition contains conceptually independent branches. The retained implementation returns $\hat{y}_A$ as the final answer. Each step receives its own retrieved context; the final step has no additional synthesis call over the union of all earlier passages.

The number of generated subquestions A may differ from the reference hop count h . One subquestion can combine several reference information needs. Several subquestions can address the same need. The evaluation therefore permits many-to-many assignments instead of requiring an alignment by position.

4.1.2 Availability, Assignment and Execution

The three evaluation objects are defined as follows.

Availability. Every annotated supporting passage occurs in at least one retrieved prefix.

Assignment. Every supporting passage has an identified owner, and each owning subquestion retrieves the passages assigned to it.

Execution. The reader produces the required intermediate answers and the final answer matches the reference under the specified answer metric.

These properties answer different questions. Evidence retrieved for a later subquestion may have been unavailable when an earlier answer was produced. Conversely, a model may answer correctly from a passage outside the annotated evidence set or from parametric knowledge. The measurements below concern recovery of benchmark evidence and reference-answer accuracy. Their relation is examined empirically rather than imposed by definition.

4.2 Retrieval Depth and Evidence Responsibility

4.2.1 The Evidence-Rank Matrix

For a recorded execution, define an $A \times h$ matrix

$$\rho_{ai} = \min\{r : d_{ar} = e_i\}. \quad (4.2)$$

The evaluation retains the first $K = 200$ ranked passages per subquestion. When e_i is absent from that prefix, its recorded rank is $+\infty$. This value denotes censoring at K , including evidence that might occur deeper in the full retrieval list. Passage matching uses the benchmark passage identity, determined from its title and text.

The matrix records where each required passage occurs for each query. It does not determine which query was responsible for that passage. That second object is introduced below.

4.2.2 Union Depth

The union depth is the smallest common prefix length that makes all annotated evidence available somewhere in the retrieval lists:

$$D_{\text{union}}(q) = \max_{1 \leq i \leq h} \min_{1 \leq a \leq A} \rho_{ai}. \quad (4.3)$$

Consequently, $D_{\text{union}}(q) \leq k$ exactly when $E(q) \subseteq \bigcup_a R_a[1 : k]$. The inner minimum allows one successful retriever to satisfy the availability requirement for a passage. The outer maximum requires every annotated passage to be present.

This definition is appropriate for measuring a pooled evidence collection. In the sequential execution of Equation 4.1, however, a passage can appear in the pooled collection without reaching the particular reader call that needed it. Union depth is therefore reported alongside an assignment-aware measurement.

4.2.3 Ownership-Masked Depth

Let $M \in \{0, 1\}^{A \times h}$ be an ownership mask, with $M_{ai} = 1$ when subquestion a is assigned responsibility for passage e_i . Define

$$D_M(q) = \begin{cases} \max_{(a,i): M_{ai}=1} \rho_{ai}, & \text{if every column of } M \text{ contains an owner,} \\ +\infty, & \text{otherwise.} \end{cases} \quad (4.4)$$

An assigned cell whose evidence is absent from the retained list also makes $D_M(q) = +\infty$. The evaluation distinguishes this retrieval censoring from an unowned column, which indicates an uncovered assignment under the predicted mask.

The row and column structure has a direct interpretation. A zero row is an idle subquestion with respect to the annotated evidence. A row with several ones assigns several evidence items to one subquestion. A zero column leaves an evidence item unowned. A column with several ones assigns that evidence to several subquestions. Idle rows contribute no retrieval obligation to Equation 4.4; their generation and reading costs remain part of the executed system.

Multiple ownership is interpreted conjunctively: each selected owner must retrieve the passage. A rule that accepts retrieval by any selected owner would use a minimum over owners and would measure a different execution requirement. The present rule preserves the interpretation of each positive cell as a specific obligation of a specific reader call.

For comparison, the all-cells maximum is

$$D_{\text{all}}(q) = \max_{a,i} \rho_{ai}. \quad (4.5)$$

It requires every subquestion to retrieve every supporting passage. Whenever the mask assigns every evidence item, the three quantities satisfy

$$D_{\text{union}}(q) \leq D_M(q) \leq D_{\text{all}}(q). \quad (4.6)$$

The first inequality follows because each selected rank is at least the smallest rank in its column. The second follows because the masked maximum ranges over a subset of all cells. A zero column is handled separately by the censoring rule in Equation 4.4.

For example, consider

$$\rho = \begin{pmatrix} 1 & 80 \\ 40 & 2 \end{pmatrix}, \quad M = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}. \quad (4.7)$$

Both union depth and ownership-masked depth equal 2, while the all-cells maximum equals 80. Each subquestion retrieves its own assigned passage near the beginning of its list. Requiring the off-diagonal passages changes the question being evaluated. With a single full-question query, the control assigns all evidence to that query, so the three depth definitions coincide.

4.2.4 Coverage and an Aggregate Depth Score

For a panel of n questions, complete-evidence coverage at depth k is

$$C_D(k) = \frac{1}{n} \sum_{j=1}^n \mathbf{1}\{D(q_j) \leq k\}, \quad (4.8)$$

where D is either D_{union} or D_M . Coverage is a question-level complete-set measurement. A question missing one required passage receives the same coverage value as a question missing several, while the retained rank matrix identifies the individual missing items.

Prompt optimisation also requires a scalar objective. The retained objective assigns logarithmic credit to finite depths:

$$v_K(d) = \begin{cases} 1 - \frac{\log d}{\log K}, & 1 \leq d \leq K, \\ 0, & d = +\infty, \end{cases} \quad J_D = \frac{100}{n} \sum_{j=1}^n v_K(D(q_j)). \quad (4.9)$$

The score is 100 when all required evidence appears at rank 1 and 0 when every question is censored or reaches depth K . Questions with unowned evidence remain in the denominator. Reporting $C_D(K)$ beside J_D distinguishes incomplete retrieval from complete retrieval at the search limit. The score summarises complete coverage and depth. Arm count and model tokens are recorded separately because Equation 4.9 does not price them.

4.3 Learning the Ownership Mask

4.3.1 Supervision from Reference Decompositions

MuSiQue associates each reference subquestion with a supporting passage. Substituting the reference intermediate answers resolves its dependencies and provides labelled subquestion–passage pairs. The associated passage is a positive ownership example; the other supporting passages of the same original question provide negative examples. These labels encode the benchmark’s reference assignment. An alternative valid decomposition can have a different assignment.

Training uses a curated set of 6 847 MuSiQue training questions. The resulting training matrix contains 36 694 cells, of which 15 452 are positive. The held-out control contains 100 questions with reference-style subquestions and known reference assignments. The retained masks provide the predictions evaluated below; the archived template history

is only partially documented. The control contains 48 two-hop, 30 three-hop and 22 four-hop questions.

The held-out question identifiers and exact resolved-subquestion–passage pairs are absent from the training set. Within the training set, questions sharing an exact repeated pair are assigned together to prevent that pair from appearing in both fitting and evaluation folds.

4.3.2 Textual and Contextual Features

For each candidate cell, a 193-dimensional feature vector describes the relation between the resolved subquestion and the supporting passage. The features combine passage-level and sentence-level ColBERT scores, lexical similarity, token alignment, title overlap and features of the subquestion before intermediate-answer substitution. ColBERT supplies contextual token representations with a late interaction score [9].

Absolute support and relative support are both represented. Background normalisation compares semantic scores against a fixed pool of 300 passages drawn from training data. Row features compare one subquestion’s support for different evidence passages. Column features compare different subquestions’ support for the same passage. Agreement features record whether different similarity measures identify the same candidate.

These comparisons are relevant to ownership: two passages may share an entity, while only one answers the relation requested by the subquestion. They also permit a cell to receive low support even when it is the best option in its row. The predictor can consequently leave a row or column empty. Retrieval rank, measured depth, reference hop position and the diagonal reference label are excluded from the feature vector. The predictor is fitted to textual assignment labels rather than to the retrieval outcomes that it will help evaluate.

The evaluation still supplies the annotated evidence passages as candidates. Thus the mask is an offline attribution instrument with known evidence to assign. Using text-based features does not by itself provide the complete evidence set for a new question at deployment.

4.3.3 Fitting, Threshold Selection and Ensemble Decision

Five grouped folds are constructed while balancing reference hop cohorts. Each rotation uses three folds to fit a weighted logistic model, one disjoint fold to select its threshold and one fold for evaluation. Regularisation is selected within the fitting portion. Standardisation parameters are estimated from the same fitting data. The retained configuration gives positive ownership examples weight 3.

For model b , let $p_b(a, i)$ be its predicted ownership score. Its threshold is obtained from the lower tail of scores assigned to positive calibration cells. If those scores in ascending order are $p_{(1)}, \dots, p_{(m)}$, the rule is

$$t_b = p_{(\lfloor (m+1)\alpha \rfloor)}, \quad \alpha = 0.01, \quad (4.10)$$

with threshold $-\infty$ when the indicated index is zero. The final mask uses a majority vote:

$$\widehat{M}_{ai} = \mathbf{1} \left\{ \sum_{b=1}^5 \mathbf{1}\{p_b(a, i) \geq t_b\} \geq 3 \right\}. \quad (4.11)$$

The lower-quantile threshold favours retaining true ownership assignments. Here α specifies the threshold-selection rule. Empirical recall, precision and complete-mask agreement quantify the resulting five-model procedure. A per-cell quantile rule and a majority vote require additional assumptions and analysis to yield a simultaneous guarantee for every assignment in a question. The results in this chapter are held-out measurements of the specified ensemble.

4.3.4 How Assignment Error Changes the Depth Measurement

Ownership prediction affects the measurement itself. Adding a positive cell adds a retrieval obligation and can increase the masked maximum. Removing a positive cell can reduce that maximum if the corresponding passage retains another owner. Removing its last owner instead censors the question. Mask errors can therefore move measured depth in either direction.

This dependence motivates evaluation at three levels: individual cells, complete arm rows and complete question masks. It also motivates freezing the attribution model when comparing decomposition prompts. A change in attribution must be distinguished from a change in the underlying retrieved lists.

4.4 Experimental Protocol

4.4.1 Systems and Retained Executions

The retrieval study uses the same 100 MuSiQue questions across the compared configurations. Its two language-model settings use GPT-4o-mini and Llama-3.3-70B-Instruct respectively for decomposition and reading. Retrieval uses HippoRAG 2 with NV-Embed-v2 embeddings and the model-specific graph and reranking configuration. HippoRAG 2 combines passage information with graph retrieval based on personalised PageRank [14]. Each reader call receives five passages; the retained rankings extend to 200 passages for depth analysis.

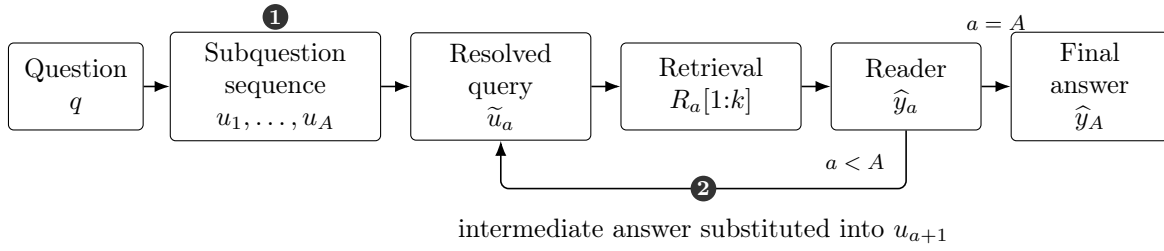
Three generated-decomposition prompts are compared with direct full-question retrieval. The baseline decomposition prompt serves as the reference generated policy, while the atomic-query prompt requests atomic single-fact queries. Both prompts use the same three demonstrations. The extended-instruction prompt uses a more detailed instruction specification without those demonstrations. These are different prompting configurations, so their comparison measures the full prompting choice rather than instruction length alone.

Two GPT-4o-mini controls use the reference decomposition. The first substitutes gold intermediate answers into downstream queries. The second substitutes the reader’s predicted intermediate answers. Both retain reference subquestions; the gold-answer condition isolates the effect of supplying correct intermediate bindings within this pipeline.

Figure 4.1 shows the execution shared by these configurations and states what each condition supplies at the two points that the comparison varies.

4.4.2 Metrics and Comparison Units

Answer quality is measured by exact match (EM) and token F1 against the reference answer and accepted aliases. Retrieval is measured by complete-evidence coverage, union



Condition	① Subquestion sequence	② Intermediate bindings
Direct	None: the full question is the only query	None: one retrieval and one reader call
Generated plan	Prompt-generated (baseline, atomic-query, extended-instruction)	Reader's predicted intermediate answers
Reference plan (predicted)	MuSiQue reference decomposition	Reader's predicted intermediate answers
Reference plan (gold)	MuSiQue reference decomposition	Gold intermediate answers (oracle-assisted)

Figure 4.1: Execution conditions in the retrieval study. The diagram is the execution shared by every condition, with the two marked points that the comparison varies: the source of the subquestion sequence and the source of the intermediate answers substituted into later subquestions. The table states what each condition supplies at those two points. Direct execution has neither, and the gold-binding condition is oracle-assisted.

depth and ownership-masked depth. The number of generated arms, retrieved passage slots and recorded total model tokens describe resource use. Per-arm passage counts charge duplicate passages again when they are sent to separate reader calls. Unique passage counts instead describe the pooled collection.

The reference-answer control records one token total across several execution branches. Per-branch costs are therefore unavailable. Generated and direct runs have separate recorded totals, which are reported per question.

Comparisons pair the same questions across configurations. Where intervals are reported, they are percentile intervals from 10 000 paired question-level bootstrap resamples. They describe variation within this development panel. Several prompt and attribution configurations were examined on the panel, so these intervals are descriptive rather than a correction for adaptive model selection. The reference-style ownership control was kept separate from weight fitting and threshold calibration; its role is a structured assignment control rather than an evaluation of arbitrary generated plans.

4.5 Results

4.5.1 Ownership Recovery on Reference-Style Arms

Table 4.1 reports the retained ownership ensemble on the reference-style control. Of 274 reference owner cells, 269 are recovered. Seven non-owner cells are selected. Precision is 97.46% and recall is 98.18%. The ensemble recovers all assignments exactly on 92 of the 100 questions. It recovers 264 of the 274 complete arm rows, which is a different unit from complete question-mask agreement.

The cohort breakdown in Table 4.2 shows the distribution of the twelve cell errors.

Table 4.1: Ownership prediction on the reference-style decomposition control. Cell metrics use 814 subquestion–passage pairs. Complete-mask agreement is measured over the 100 questions.

Measurement	Value
True positives / false negatives	269 / 5
False positives / true negatives	7 / 533
Precision	97.46%
Recall	98.18%
Cell accuracy	98.53%
AUROC of mean model scores	0.9977
Exact arm rows	264/274
Exact question masks	92/100

Four-hop questions contain three of the five missed owners. Because several cells belong to the same question, cell totals are reported with their question counts rather than interpreted as independent question outcomes.

Table 4.2: Ownership errors by reference hop count. TP, FN, FP and TN count individual cells, while n counts questions.

Reference hops	n	Cells	TP	FN	FP	TN
2	48	192	95	1	4	92
3	30	270	89	1	0	180
4	22	352	85	3	3	261

On the gold-answer retrieval traces, the exact reference mask yields complete assigned-evidence coverage of 100% at depth 200 and a score of 86.47. Using the learned ownership mask gives 97% coverage and a score of 84.84. This comparison isolates the effect of replacing the reference assignment with its prediction on those fixed lists.

4.5.2 Sensitivity to the Attribution Model

Table 4.3 compares two attribution models on the same retrieval traces: the earlier token-multiview model and the retained ownership-multiview model. The baseline generated decomposition receives scores of 52.32 and 72.19 respectively. The analogous atomic-query values are 52.81 and 66.64. These changes arise from the assigned cells, with the passages and ranks held fixed.

The reference-style control supports the use of the retained predictor for recovering known assignments. For generated arms, assignments can legitimately be fused or shared. The score differences in Table 4.3 therefore establish sensitivity to attribution rather than a labelled accuracy comparison on generated plans. Prompt comparisons in Chapter 5 keep the current ownership model fixed.

4.5.3 Evidence Availability and Final Answers

For readability, $C_U(k)$ denotes the union coverage $C_{D_{\text{union}}}(k)$ in the result tables. Table 4.4 reports the direct and generated-decomposition executions. With GPT-4o-mini, the baseline decomposition raises complete union coverage at five passages per arm from

Table 4.3: Masked-depth measurements under two attribution models on fixed GPT-4o-mini retrieval traces. Coverage is complete assigned-evidence coverage at $K = 200$; score is J_{D_M} from Equation 4.9. The last row uses the exact reference assignment.

Decomposition	Attribution	Coverage (%)	Score
Baseline	Token-multiview	78	52.32
Baseline	Ownership-multiview	87	72.19
Atomic-query	Token-multiview	77	52.81
Atomic-query	Ownership-multiview	81	66.64
Reference, gold bindings	Token-multiview	88	59.86
Reference, gold bindings	Ownership-multiview	97	84.84
Reference, gold bindings	Reference assignment	100	86.47

45% to 76%. EM changes from 40% to 52% and F1 from 52.64 to 59.00. With Llama-3.3-70B, complete union coverage changes from 46% to 79%, while EM changes from 42% to 43% and F1 from 52.30 to 51.37.

Table 4.4: Recorded execution results on 100 MuSiQue questions. EM and F1 are percentages. $C_U(5)$ is complete union coverage at five passages per arm. Arms and tokens are per-question means. Retrieved evidence and final answers are separate outcomes.

Model	Prompt	EM	F1	$C_U(5)$	Arms	Tokens
GPT-4o-mini	Direct	40	52.64	45	1.00	4 369
	Baseline	52	59.00	76	2.79	12 763
	Atomic-query	52	59.66	72	2.66	11 898
	Extended-instruction	50	60.39	74	2.61	13 881
Llama-3.3-70B	Direct	42	52.30	46	1.00	4 530
	Baseline	43	51.37	79	3.00	14 069
	Atomic-query	45	53.54	78	3.22	14 789
	Extended-instruction	47	55.94	75	2.84	15 271

The paired bootstrap interval for the baseline decomposition’s EM change is $[2, 22]$ percentage points for GPT-4o-mini and $[-8, 10]$ for Llama-3.3-70B. These measurements show why retrieval and answer metrics are both needed. Improved pooled availability can coexist with a small change in final answers, because the sequence must still use the retrieved evidence and propagate useful intermediate states.

The per-arm comparison also changes the total retrieval expenditure. The baseline GPT-4o-mini decomposition retrieves a mean of 13.95 passage slots across its reader calls, compared with 5 for direct retrieval. Its mean pooled collection contains 11.35 distinct passages. Thus the coverage increase in Table 4.4 combines a change in query formulation with more passage slots and more reader calls. Section 4.6 compares fixed lists at a common total passage budget.

4.5.4 The Role of Intermediate Bindings

The two reference-decomposition controls use the same subquestion templates and reader but change the answers substituted into later queries. Table 4.5 reports their outcomes. Gold substitution yields EM 64% versus 49% under predicted substitution, with a paired bootstrap interval of $[8, 22]$ percentage points for the difference. Complete union coverage at five passages per arm changes from 80% to 92%.

Table 4.5: Reference-decomposition controls with GPT-4o-mini. Gold bindings supply reference intermediate answers when constructing downstream queries. Predicted bindings use the reader’s earlier outputs.

Intermediate substitution	EM (%)	F1	$C_U(5)$ (%)
Predicted answers	49	57.50	80
Gold answers	64	70.73	92

The intervention changes downstream query text and consequently can change both retrieval and answer generation. Its effect includes error propagation through those two stages. Supplying gold intermediate answers defines an oracle-assisted control. The measured gap identifies a role for intermediate state quality within the reference-chain pipeline; it is not a gain achieved by an operational binding policy.

4.6 Depth Under a Total Passage Budget

4.6.1 Equal-Allocation Replay

A common per-arm depth gives more total passage slots to longer decompositions. To separate these quantities, consider a per-question ceiling $B \leq AK$ distributed among the A recorded retrieval lists. Equal allocation uses the whole allowance and assigns

$$k_a(B) = \left\lfloor \frac{B}{A} \right\rfloor + \mathbf{1}\{a \leq B \bmod A\}, \quad \sum_{a=1}^A k_a(B) = B. \quad (4.12)$$

The remainder is allocated in execution order. A passage slot is charged each time it occurs in an arm prefix, including duplicates across arms. Complete evidence availability is then evaluated in $\bigcup_a R_a[1 : k_a(B)]$.

This allowance counts selected passage slots. Equal slot counts can produce different numbers of unique passages and different token totals, since passages can recur across arms and vary in length. Retrieving the ranked lists, producing the queries and reading the contexts incur separate computational costs.

These are fixed-list replays. The original intermediate answers and later queries were generated using five passages per reader call. Changing a prefix in the replay leaves those earlier answers and later queries unchanged. The result measures the coverage of the recorded lists at a specified slot budget. An end-to-end policy evaluated at that budget would regenerate intermediate answers and dependent queries under its actual contexts.

Per-question and average budgets. Under a hard ceiling B for each question, the largest feasible common depth within a question is $\lfloor B/A(q) \rfloor$. Complete-evidence availability is non-decreasing in every prefix length, so this choice weakly dominates smaller equal depths. Equation 4.12 also spends the integer remainder. A single depth shared across the entire panel is a weaker comparator when arm counts vary: feasibility limits it to $\lfloor B/\max_q A(q) \rfloor$. Unequal within-question allocation can improve on the balanced comparator by placing more slots on deeper arms. An average budget instead permits transfers between questions, including transfers that retain positive allocations for every question. That is a different resource constraint from the per-question comparisons here.

4.6.2 A Fixed-List Oracle Allocation

The rank matrix also permits an exact comparison with the least expensive prefix allocation for its recorded lists:

$$B^*(q) = \min_{k_1, \dots, k_A \in \{0, \dots, K\}} \left\{ \sum_a k_a : E(q) \subseteq \bigcup_a R_a[1 : k_a] \right\}. \quad (4.13)$$

The value is $+\infty$ when the retained lists jointly miss annotated evidence. For finite cases, it can be computed by dynamic programming over subsets of evidence. Let $S_a(k)$ denote the evidence indices found in arm a ’s first k passages. Starting from $F_0(\emptyset) = 0$, the update is

$$F_a(T \cup S_a(k)) \leftarrow \min\{F_a(T \cup S_a(k)), F_{a-1}(T) + k\}. \quad (4.14)$$

Unreachable states have value $+\infty$, and $B^*(q) = F_A(\{1, \dots, h\})$. Only zero and prefix lengths at which an additional evidence item appears need to be considered: a longer prefix with the same evidence set adds cost without changing that state. With at most four annotated hops in this panel, the subset state space is small.

The oracle uses gold evidence identities to choose its allocation. Moreover, all recorded queries are available even when their selected prefix length is zero. The objective charges passage slots but leaves the generation of those queries and their intermediate states outside the optimisation. It is an exact minimum for the fixed-list problem and an optimistic reference for a system that must construct and execute the queries. It is not an achieved runtime or token saving.

4.6.3 Allocation Headroom on the Recorded Lists

At $B = 15$, equal allocation recovers complete evidence for 76 questions under the baseline GPT-4o-mini decomposition and 78 under its Llama counterpart. The corresponding oracle allocations reach 85 and 87. Table 4.6 reports the same comparison for the other prompts and direct retrieval.

Table 4.6: Complete evidence coverage (%) at a total budget of 15 passage slots on fixed recorded lists. The oracle chooses prefixes using gold evidence and is a reference allocation rather than a deployed policy. Each row contains 100 questions.

Model	Prompt	Equal allocation	Oracle allocation
GPT-4o-mini	Direct	66	66
	Baseline	76	85
	Atomic-query	73	85
	Extended-instruction	75	83
Llama-3.3-70B	Direct	65	65
	Baseline	78	87
	Atomic-query	75	88
	Extended-instruction	75	85

The baseline decompositions retain a coverage advantage over direct retrieval in this replay after the total slot budget is matched. The additional nine questions recoverable by oracle allocation in each model setting identify uneven depth requirements across arms. This result motivates evaluation of observable allocation policies. Their value must be

measured after accounting for the decisions and intermediate answers needed to obtain the lists.

4.7 A Cross-Benchmark Fixed-Trace Study

4.7.1 Retrieval Regime and Evaluation Population

A complementary BM25 study applies the passage-slot accounting to MuSiQue, 2Wiki-MultiHopQA [?] and HotpotQA [?]. It uses separate closed corpora containing 11 656, 10 426 and 9 811 passages, respectively. Supporting passages are identified from each benchmark’s annotations. The metric is complete annotated-evidence availability in the selected union; it requires the evidence set but no estimated ownership mask. The ownership measurement remains relevant when the question is which arm has the evidence needed for its own decision.

Each source panel contains 1 000 questions. Saved generated-plan traces are available for 824 MuSiQue, 977 2Wiki and 996 HotpotQA questions. The full-question control is restricted to the same retained IDs within each dataset. Thus the comparisons are paired on available traces, with 176, 23 and 4 source questions outside their respective denominators. These coverage values are conditional on obtaining a retained trace; they do not represent success rates for the entire original intake. The source panels also have historical project exposure, so these results are development evidence separate from the frozen comparison in Chapter 5.

The sequential traces use generated subquestions. A GPT-4o-mini reader answers each arm from five BM25 passages, and those predicted answers resolve later references. Up to 200 ranks per arm are saved for replay. This is a different retriever and execution implementation from the HippoRAG experiments above. Every allocation below acts on those saved lists. Its slot budget excludes the calls and contexts used to generate the intermediate answers.

Selected slots, unique pooled passages and reader tokens remain distinct. Two allocations with the same $\sum_a k_a$ can have different overlap between arms and hence different deduplicated context sizes. A subsequent reader over the pooled union is an additional policy; its answer quality must be measured separately from the coverage frontiers reported here.

4.7.2 Matched Passage-Slot Frontiers

Table 4.7 compares full-question retrieval, balanced allocation from Equation 4.12, and the fixed-list oracle at $B = 8$. The balanced allocation spends the integer remainder in execution order. The saved plans contain at most seven arms on MuSiQue and HotpotQA and six on 2Wiki, so this budget can allocate at least one slot to every arm.

The oracle recovers 12.4, 19.3 and 7.4 additional percentage points over balanced allocation on MuSiQue, 2Wiki and HotpotQA, respectively. These are opportunities within the existing rankings. They preserve the recorded queries and reveal how unequal evidence ranks constrain a balanced split. The gain from generating better queries is outside this oracle’s policy class.

The subset dynamic program in Equation 4.14 independently reproduces the saved oracle coverage on all three retained panels at the checked budgets. For a fixed panel, its

Table 4.7: Complete annotated-evidence coverage at a ceiling of eight selected passage slots. BM25 results use fixed predicted-binding traces; the oracle uses gold evidence ranks. Counts are retained questions from each original 1,000-question panel.

Dataset	n	Full question	Balanced	Oracle
MuSiQue	824	0.136	0.413	0.536
2Wiki	977	0.315	0.688	0.881
HotpotQA	996	0.637	0.762	0.836

Table 4.8: MuSiQue BM25 retrieval regimes. Coverage is measured on each retained panel under balanced slot allocation. The reference row uses a different plan source and a larger panel.

Query regime	n	Coverage at $B = 8$	Coverage at $B = 200$
Reference plan, gold bindings	1 000	0.595	0.962
Generated plan, predicted bindings	824	0.413	0.733
Generated plan, paraphrased references	824	0.109	0.397

frontier is

$$C^*(B) = \frac{1}{n} \sum_{q=1}^n \mathbf{1}\{B^*(q) \leq B\}. \quad (4.15)$$

Questions whose lists jointly miss a required passage remain uncovered for every allocation. Zero-depth arms are permitted by the oracle, with the cost of constructing their saved queries kept outside the objective.

A complementary reading fixes coverage and compares the first tested budget that reaches it. On the MuSiQue sequential panel, balanced allocation first reaches coverage 0.50, 0.60 and 0.70 at 16, 32 and 128 slots. The oracle reaches the same targets at 8, 16 and 48 slots. These values are minima over the reported budget grid, rather than exact interpolated crossings. They quantify allocation headroom and do not constitute achieved token or runtime reductions.

Generated arm count also differs from benchmark reasoning depth. Mean arm counts are 2.53, 2.79 and 2.10 on the retained MuSiQue, 2Wiki and HotpotQA panels. More arms create more allocation choices and can increase the cost of a common per-arm depth. They can also reflect redundant splitting or a prompt preference. An association between allocation headroom and arm count therefore concerns plan structure, rather than establishing that semantically harder questions obtain greater savings.

4.7.3 Reference and Generated Query Regimes

The MuSiQue artifacts support three retrieval regimes. Reference plans use the benchmark decomposition and gold intermediate answers. Sequential plans use generated subquestions and the reader’s predicted answers. Paraphrased queries replace a reference with a noun phrase derived from the referenced subquestion, without reading an intermediate answer. Table 4.8 reports their balanced coverage at two total slot budgets.

The sequential and paraphrase archives contain the same 824 question IDs and the same arm counts. Their different rankings show that the way intermediate references are turned into query text materially changes evidence availability. Reference plans provide a separate oracle-assisted comparison: they change both the source of the plan and its

bindings, and their full reported panel has a different size. The three-row comparison therefore does not isolate a pure binding intervention. The controlled reference-chain comparison earlier in this chapter supplies the narrower evidence about predicted versus gold intermediate answers.

The distinction also constrains interpretation of allocation. At $B = 200$, the sequential fixed-list oracle covers 0.795 of the retained MuSiQue questions, compared with balanced coverage 0.733. Both results concern the selected prefixes of queries already produced with five passages per intermediate reader. A changed operational budget can alter those answers and later rankings, which requires re-execution to evaluate.

4.8 Discussion and Implications for Optimisation

The evaluation decomposes a retrieval plan into measurable obligations. Union depth measures pooled evidence availability. The ownership mask localises that evidence to the subquestion expected to use it. Answer metrics then measure the outcome of executing the sequence. These distinctions prevent a shallow union from being interpreted as a fully supported reasoning chain and prevent a masked-depth change caused by attribution from being attributed to improved retrieval.

The ownership study provides a working measurement model. It recovers the reference assignment on 92 of 100 complete reference-style masks while retaining high cell recall and precision. Its empirical scope is reference-style arms with known supporting passages. Generated plans can change the semantic structure through fusion, splitting, omission or incorrect dependency substitution. Chapter 5 evaluates controlled transformations of reference plans to measure sensitivity to several of these changes. Transfer to naturally generated plans requires independently labelled assignments from that regime.

The answer experiments establish the importance of execution state. Under reference decompositions, substituting gold intermediate answers changes both retrieval coverage and final accuracy. An unowned passage in the predicted mask can therefore have several explanations: a missing information need, an altered query following an earlier error, or a classifier disagreement. Feedback for optimisation should retain those distinctions. A predicted attribution diagnosis is evidence to inspect, rather than a verified causal explanation of the failure.

The resource measurements distinguish retrieval depth, total passage slots and model tokens. The masked objective rewards shallow completion of assigned evidence; the fixed-budget replay measures allocation efficiency on existing lists. A prompt that raises the former can still create more arms or consume more tokens. Chapter 5 consequently evaluates the optimisation objective together with complete-evidence coverage, answer quality and execution cost.

The complementary BM25 study extends the fixed-slot analysis to three benchmarks and larger retained panels. Its oracle gaps motivate observable allocation policies, while its query-regime comparison establishes the need to document how dependent subquestions were resolved. Those results retain their closed-corpus, fixed-trace and available-case scope alongside the separate attribution and answer measurements.

The current system applies decomposition to every question in these experiments. The difficulty probe of Chapter 3 supplies a separate measurement and is not used to select or route these executions. Connecting a pre-answer difficulty estimate to a decomposition policy requires its own cost and answer evaluation.

Conclusion

This chapter addressed the evaluation component of the retrieval-efficiency question posed in Section 2.6.2. It defined complete evidence availability and ownership-masked retrieval depth, trained a textual ownership predictor and evaluated its assignments on reference-style decompositions. The predictor recovered 269 of 274 owner cells and 92 complete question masks out of 100.

The retrieval study showed that evidence availability, assignment and answer execution vary separately. Generated decompositions increased complete pooled retrieval coverage, while the answer gains depended on the model and execution state. A fixed-budget analysis also identified room to allocate passage slots more selectively across the recorded arms. These findings support an evidence-aware optimisation objective while specifying the additional answer and cost measurements required to assess its practical effect. Chapter 5 uses this evaluation to study decomposition prompt search and learned passage allocation.

Chapter 5

Optimising Question Decomposition and Evidence Allocation

Introduction

Chapter 4 introduced an evidence-aware evaluation of a generated decomposition. The ownership mask assigns supporting passages to subquestions, and the resulting depth score measures how early those passages can be retrieved by their assigned queries. This chapter uses that score to optimise the instruction that generates the decomposition. It addresses the question posed in Section 2.6.2: can the generation strategy preserve evidence coverage while reducing retrieval depth?

The completed experiment applies an OPRO-style search to one system instruction. It evaluates twenty-four proposed instructions and the initial instruction on the same hundred MuSiQue questions. The selected instruction raises the masked depth score from 66.30 to 67.20, with unchanged exact match of 44%. These measurements describe prompt selection on the development panel. They also motivate a distinction between improving a retrieval objective and improving the answers produced by the complete system.

The chapter presents two complementary interventions. Prompt search changes the questions issued by the system. Learned allocation changes the passage prefix selected from each arm of a recorded execution. The completed allocation study uses predicted-binding BM25 traces on MuSiQue, 2Wiki and HotpotQA. At a budget of eight passage slots, its largest observed coverage increase is 6.3 percentage points on 2Wiki. These are retrieval-coverage measurements on retained traces, distinct from the pilot’s final-answer measurements. A paired 700-question MuSiQue experiment also evaluates the answers produced from balanced and segment-policy pooled contexts. The later unit-increment policy is evaluated separately on retrieval coverage.

The chapter then specifies an evidence-feedback extension that separates observed retrieval and dependency information from estimated ownership. A comparison with answer-only and information-matched raw feedback is defined on frozen optimisation, selection and confirmation splits. The extension has an implemented feedback and search interface; its comparative answer-quality evaluation remains prospective. The empirical conclusions concern the completed OPRO pilot, fixed-trace allocation study and controlled ownership stress tests.

5.1 The Optimisation Problem

5.1.1 An Instruction Defines an Executable Policy

Let p denote the decomposer’s system instruction and let q be an input question. With fixed demonstrations and generation settings, the decomposer produces an ordered plan

$$\pi_p(q) = (u_1, \dots, u_{A_p(q)}), \quad (5.1)$$

where $A_p(q)$ is the number of subquestions. A reference $\#i$ in u_a refers to the answer of an earlier arm, so valid references satisfy $i < a$. The executor substitutes its predicted intermediate answers, retrieves passages and reads them for each arm in sequence. The answer of the last arm is the final answer $\hat{y}_p(q)$.

This interface constrains useful prompt changes. Each arm must be phrased as a retrieval question, references must be executable in order, and the final arm must target the answer requested by q . A plan that retrieves relevant facts but ends with an intermediate entity can have adequate evidence coverage and an incorrect final answer. The optimisation therefore acts on a program of retrieval and reading steps, rather than on a list of paraphrases alone.

The historical experiment changes p while keeping the demonstrations, retriever, reader and ownership estimator fixed. It uses one generated plan per question. The six instructions proposed in a search round are six policy candidates, not six plans combined at inference time.

5.1.2 The Retained Retrieval Objective

For each executed plan, let $D_{M,p}(q)$ be the ownership-masked depth defined in Chapter 4. It is the greatest passage rank among the assigned arm–evidence pairs, with infinite depth when required evidence has no predicted owner or falls outside the retained ranked lists. With cutoff $K = 200$, the search objective is

$$J_M(p; \mathcal{D}) = \frac{100}{|\mathcal{D}|} \sum_{q \in \mathcal{D}} v_K(D_{M,p}(q)), \quad v_K(d) = \begin{cases} 1 - \frac{\log d}{\log K}, & 1 \leq d \leq K, \\ 0, & d = +\infty. \end{cases} \quad (5.2)$$

All questions remain in the denominator. The objective rewards early complete retrieval under the predicted assignment and gives zero credit to a censored question. Its companion coverage statistic is

$$C_M(K; p) = \frac{1}{|\mathcal{D}|} \sum_{q \in \mathcal{D}} \mathbf{1}\{D_{M,p}(q) \leq K\}. \quad (5.3)$$

Depth K receives zero logarithmic credit but counts as finite coverage. Reporting both quantities distinguishes complete evidence near the search limit from incomplete evidence.

The score depends on the ownership estimator. An assignment error can change which ranks enter the maximum, as discussed in Chapter 4. The estimator is therefore frozen during search. Evidence coverage computed directly from the gold passage set remains a separate diagnostic.

Table 5.1: Configuration of the completed prompt-search pilot.

Component	Setting
Evaluation panel	The same 100 MuSiQue questions for every candidate
Initialisations	One initial instruction
Search rounds	Four, with six new instructions per round
Candidate total	25, including the initial instruction
Proposal model	GPT-4o, temperature 1.0
Execution model	Llama-3.3-70B, one plan per question
Reader context	Five passages per executed arm
Retrieval scoring	Up to 200 retained ranks per arm
Search reward	Ownership-masked logarithmic depth score
History in feedback	At most 20 prior instructions and their rounded scores
Example feedback	Three cases from the current leading instruction

5.1.3 Answer Quality and Resource Use

The end-to-end outcomes are exact match and token F1 against the benchmark reference answers. For a fixed evaluation panel, define

$$J_{\text{F1}}(p) = \frac{1}{|\mathcal{D}|} \sum_{q \in \mathcal{D}} \text{F1}(\hat{y}_p(q), y_q). \quad (5.4)$$

The retrieval objective in Equation 5.2 and the answer objective in Equation 5.4 measure different properties. Retrieved evidence must still be read, intermediate answers must bind correctly, and the last arm must produce the requested answer.

The historical reader consumes five passages per arm. Its nominal passage-slot allowance is consequently $5A_p(q)$, with repeated passages charged to each arm that reads them. Unique passages, reader tokens and model calls are additional cost measurements. The masked-depth objective contains no direct penalty for arm count, so a higher objective value alone establishes neither lower total reading cost nor higher answer accuracy.

5.2 The Completed OPRO Search

5.2.1 Search Configuration

OPRO uses a language model to propose solutions from a history of candidates and their measured scores [?]. Here, a candidate is the text of the decomposer’s system instruction. GPT-4o proposes instructions, and the Llama-3.3-70B pipeline executes them on the development questions. Table 5.1 records the retained configuration.

The executor uses temperature-zero generation with cached model calls. Exact duplicate instruction hashes are filtered. The retained leaderboard contains twenty-five successfully evaluated candidates and zero rejected candidates. This amounts to 2500 candidate–question evaluations. That count describes logical evaluations; cached calls can reduce the number of billable model requests.

The baseline is the seed instruction and its executions recorded within this search. The separately recorded prompt variants in Chapter 4 provide context for the pipeline, rather than substituting for this paired baseline.

5.2.2 Proposal and Selection Loop

The search begins by executing the initial instruction on all hundred questions. At each round, the optimizer receives the retained instruction history, the corresponding aggregate depth scores and three examples from the current leader. It proposes six replacement instructions. Each is executed through the same retrieval and reading pipeline, assigned an ownership mask and scored using Equation 5.2. The updated leaderboard retains the highest-scoring instruction across all completed rounds.

The full-precision score determines the leaderboard order, while the meta-prompt displays integer-rounded scores. Twelve of the twenty-five candidates are displayed in the same score-64 bucket. This reduces the resolution of the numerical information available to the proposer, even though their stored scores remain distinct.

The initial and selected instructions share the same planning policy: use complete natural-language questions, preserve independent branches, require backward references and split compound lookups into simpler arms. The selected instruction adds an explicit check that the necessary facts of the original question are covered. These are observed differences in instruction text. The experiment changes several clauses together, so the measured score change cannot be assigned to that coverage clause alone.

5.2.3 The Feedback Supplied to the Optimizer

The historical feedback prioritises questions with predicted unowned evidence, then questions with the greatest masked depth. It selects the unowned cases in stored question order. This gives the optimizer examples of poorly scored executions, but repeatedly drawing from the current leader and a fixed ordering can concentrate attention on a small part of the panel.

The example description attributes failures to facts that the plan failed to ask for. That interpretation is stronger than the underlying measurements. A passage may be assigned to an appropriate query yet appear at a large retrieval rank. Conversely, an unowned passage is an estimator output whose correctness depends on the ownership model. Both cases can lower the depth score, while suggesting different changes to the instruction. This motivates the typed feedback in Section 5.5.

5.3 Results of the Pilot

5.3.1 Progress Across Search Rounds

Table 5.2 separates the best proposal in each round from the best instruction retained so far. The overall winner appears in the second round. The third and fourth rounds produce lower-scoring candidates, while the stored best score remains 67.20.

The increase from the initial instruction is 0.9025 score points. A plateau over the final two rounds describes this trajectory and its proposal budget. Broader search, different initial instructions and alternative feedback are separate experimental choices. The four-round trajectory does not determine their attainable performance.

Table 5.2: Masked-depth score during the completed search. The retained best includes all earlier rounds. Values are on the shared development panel.

Stage	New candidates	Best in stage	Retained best
Initial instruction	1	66.3017	66.3017
Round 1	6	66.5975	66.5975
Round 2	6	67.2042	67.2042
Round 3	6	65.2557	67.2042
Round 4	6	66.0482	67.2042

Table 5.3: Initial and selected instructions on the 100 development questions. F1 is displayed on a 0–100 scale. Passage slots are the five-per-arm allowance derived from the observed arm count.

Measurement	Initial	Selected	Change
Masked-depth score	66.3017	67.2042	+0.9025
Finite masked coverage (%)	80	80	0
Exact match (%)	44	44	0
Token F1	52.3682	52.4847	+0.1165
Mean generated arms	3.00	3.13	+0.13
Mean passage-slot allowance	15.00	15.65	+0.65

5.3.2 The Selected Instruction

Table 5.3 compares the selected instruction with the seed on the quantities available for both. Finite masked coverage remains at 80%. Thus the higher aggregate score is not accompanied by a higher aggregate count of questions with finite masked depth. Question-level successes can still change identity.

Exact match is unchanged at 44%: two questions change from incorrect to correct and two change in the opposite direction. Mean F1 increases by 0.1165 points on the 0–100 scale. The selected instruction generates 0.13 additional arms per question, corresponding to a 4.3% increase in the mean passage-slot allowance. Its measured benefit is therefore specific to the masked-depth score, with broadly similar final-answer quality and a slightly larger nominal reading allowance.

A paired bootstrap over the hundred questions gives a descriptive 95% interval of $[-1.80, 3.90]$ points for the masked-depth-score difference and $[-3.60, 4.00]$ points for the F1 difference. The exact-match difference has an interval of $[-4, 4]$ percentage points. These intervals condition on the selected pair of instructions. The same panel supplied feedback and selected the winner, so the intervals describe sensitivity to the sampled questions; they are not an independent assessment of selection generalisation.

5.3.3 The Optimisation Objective and the Answer Objective

Across the twenty-five evaluated instructions, the Spearman correlation between masked-depth score and mean answer F1 is 0.413. The candidates are adaptively related and share their evaluation questions, so this is a description of the retained candidate set. It is useful for inspecting whether the objective ranks these instructions like answer quality does.

The highest-F1 candidate occurs in the third round. It achieves F1 53.88 and exact

Table 5.4: Different objectives select different candidates from the same development search. The F1-leading candidate is identified retrospectively.

Instruction	Depth score	F1	Exact match (%)
Initial	66.30	52.37	44
Highest depth score	67.20	52.48	44
Highest answer F1	64.10	53.88	46

match 46%, with masked-depth score 64.10. The depth-selected winner has F1 52.48 and exact match 44%, with score 67.20. Table 5.4 shows the resulting reversal. The highest-F1 candidate is a retrospective comparator, not a second independently selected test winner.

This result supports reporting retrieval quality and answer quality together. An evidence-depth objective remains appropriate when the task is to bring assigned evidence earlier in the ranked lists. For a claim about improved question answering, final answers provide the direct selection criterion. Objective choice and feedback choice should therefore be varied separately.

5.4 Learned Allocation on Fixed Retrieval Traces

5.4.1 The Allocation Intervention

Prompt optimisation changes a plan and its resulting rankings. Allocation holds those rankings fixed and chooses how much evidence to retain from each arm. For a question with A recorded arms, let k_a be the prefix length selected from arm a . Its passage-slot budget is

$$\sum_{a=1}^A k_a \leq B. \quad (5.5)$$

Complete coverage means that every annotated supporting passage occurs in at least one selected prefix. This is the union criterion from Chapter 4. An occurrence in two arms is charged twice in Equation 5.5, although a pooled reader may later deduplicate it.

The learned policy is evaluated on BM25 rankings generated by a sequential execution. Each subquestion is answered using its first five retrieved passages; that predicted answer resolves references in later subquestions. The run retains 200 ranks per arm. Allocation replays these saved lists. Changing B therefore preserves the intermediate answers and downstream queries from the five-passages-per-hop trace-generation policy. The experiment measures context selection conditional on that policy, rather than a fresh budget-dependent execution of the reasoning chain.

The balanced comparator assigns $\lfloor B/A \rfloor$ slots to every arm and distributes the remainder to the earliest arms. The learned rule also starts with one slot per arm. Both comparisons use the same total budget when $B \geq A$. The retained traces contain at most seven arms, so the budgets 8, 16 and 32 reported below satisfy that condition. Zero-depth arms are allowed in the gold-informed oracle from Chapter 4; it is a reference with a broader feasible set than these floor-constrained policies.

5.4.2 Predicting Per-Arm Depth Requirements

Training assigns each supporting passage to the arm that ranks it earliest, breaking ties by arm order. An arm’s target depth d_a is the largest rank among its assigned passages, with target one for an empty assignment. If any required passage is absent from every retained list, all arm targets for that question are censored beyond the maximum depth. This convention couples the training labels across arms.

For each threshold in

$$\mathcal{K} = \{1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64, 96, 128, 200\},$$

a binary gradient-boosted classifier predicts

$$\widehat{S}_a(k) = \widehat{\Pr}(d_a \leq k \mid x_a). \quad (5.6)$$

The implementation fits separate threshold classifiers and then applies a running maximum over their predictions. This creates non-decreasing cumulative curves. It is an upward monotone envelope, with probability calibration remaining a separate property to assess.

Features describe arm text, position, arm count and the first ten BM25 scores. The score features include their spread, concentration, selected gaps and decay. They are computed from the saved rankings before prefix selection. Their acquisition is outside the passage-slot accounting in Equation 5.5. The reported budget therefore measures selected context, rather than total retrieval work, latency or model cost. On sequential traces, later resolved query text also depends on the earlier five-passage reader calls.

5.4.3 Segment and Unit-Increment Allocation

The allocation score is the separable surrogate

$$\max_{k_a \geq 1, \sum_a k_a \leq B} \sum_{a=1}^A \log \widehat{S}_a(k_a). \quad (5.7)$$

It treats satisfaction of the assigned arm requirements as conditionally independent. Union coverage remains the evaluation criterion: evidence retrieved by another arm can satisfy the question even when the assigned arm misses it.

The segment policy forms an upper concave envelope of each log curve. It takes affordable whole envelope segments in descending gain-per-slot order, then distributes unused slots to arms with the largest remaining probability mass. This fills the budget, subject to the retained-depth cap. It is a heuristic for Equation 5.7, including when the original log curves are concave. For example, on depths $\{1, 2, 3\}$, log curves $(-4, -3.5, -3)$ and $(-3, -2, -1)$ with $B = 3$ produce allocation $(2, 1)$ and score -6.5 , while $(1, 2)$ achieves -6 . The remainder rule chooses the lower-confidence arm rather than the larger next marginal gain.

The unit-increment policy instead allocates one slot at a time. Let h_a be the piecewise-linear upper concave envelope of $\{(k, \log \widehat{S}_a(k)) : k \in \mathcal{K}\}$. Starting from $k_a = 1$ for every arm, select an uncapped arm maximising

$$\Delta_a(k_a) = h_a(k_a + 1) - h_a(k_a) \quad (5.8)$$

and increment its depth. The retained cap is 200. Once every marginal gain is zero, additional slots can be assigned to any uncapped arm without changing the objective. With the non-decreasing predicted curves, this rule solves

$$\max_{k_a \in \{1, \dots, 200\}, \sum_a k_a \leq B} \sum_{a=1}^A h_a(k_a), \quad B \geq A. \quad (5.9)$$

Concavity makes each arm’s successive unit gains non-increasing. Every allocation corresponds to taking a prefix of those gains from each arm. Selecting the greatest available gain at each step therefore selects the greatest remaining gain overall while preserving the prefix constraint. An exchange of any smaller selected gain with a larger available gain cannot reduce the objective. Repeating this exchange gives the greedy allocation and establishes its optimality for Equation 5.9.

This exactness concerns the interpolated envelope. The envelope can lie above the original log curve even at a measured depth. For example, on depths $\{1, 2, 3\}$, curves $(-4, -3.9, -1)$ and $(-3, -2, -1)$ with $B = 3$ give unit allocation $(2, 1)$. Its original log score is -6.9 , whereas $(1, 2)$ achieves -6 . The interpolated and original objectives therefore remain distinct, and both differ from complete union coverage.

An exact finite-grid alternative is available through dynamic programming. Let $V_a(b)$ be the best surrogate score for the first a arms with at most b slots. Then

$$V_a(b) = \max_{k \in \mathcal{K}, k \leq b} \left\{ V_{a-1}(b - k) + \log \hat{S}_a(k) \right\}, \quad (5.10)$$

with $V_0(b) = 0$ for $b \geq 0$ and infeasible states equal to $-\infty$. This costs $O(AB|\mathcal{K}|)$ and optimises the declared discrete surrogate. The coverage measurements below compare the segment and unit-increment policies; dynamic programming remains an alternative for the original grid objective. Surrogate optimisation and the gold-informed union oracle solve different allocation problems.

5.4.4 Evaluation Population and Cross-Validation

The saved sequential evaluations retain 824 MuSiQue, 977 2Wiki and 996 HotpotQA questions from the corresponding thousand-question panels. The missing trace counts are 176, 23 and 4, respectively. Results condition on the available traces, whose inclusion can depend on successful generation and execution. Chapter 4 records this population and the fixed-trace baselines.

Five-fold cross-validation splits by question. Every arm of a held-out question is predicted by classifiers trained on the other folds, including the feature standardisation. Shared atomic subproblems and supporting documents may recur across folds. Method choices were developed on these panels, so the resulting point estimates describe a development study rather than locked confirmation. They are distinct from the source-component-separated evaluation in Section 5.6.

Reference-decomposition results use benchmark subquestions and form a separate regime. Tables 5.5 and 5.6 use only generated sequential traces with predicted bindings. The generated-plan assignment labels are obtained from earliest passage ranks rather than an assumed alignment between arm indices and benchmark hop indices.

Table 5.5: Allocation on fixed BM25 traces. Segment denotes the segment heuristic and Unit the unit-increment optimiser of the concave interpolant. The budget counts selected slots, excluding trace generation and shallow-score acquisition. Differences are percentage points relative to balanced allocation.

Dataset	Budget	Balanced	Segment	Unit	Δ_{Unit} (pp)
MuSiQue	8	0.413	0.445	0.460	+4.7
	16	0.528	0.541	0.546	+1.8
	32	0.607	0.604	0.609	+0.2
2Wiki	8	0.688	0.746	0.751	+6.3
	16	0.864	0.884	0.892	+2.8
	32	0.926	0.932	0.932	+0.6
HotpotQA	8	0.762	0.763	0.770	+0.8
	16	0.859	0.858	0.856	-0.3
	32	0.907	0.904	0.904	-0.3

Table 5.6: Smallest tested passage-slot budget reaching a coverage target on the fixed generated sequential traces, using the segment policy. The oracle uses gold passage ranks and permits zero-depth arms. Ratios compare discrete operating points.

Dataset	Target	Balanced	Segment	Oracle	Balanced/segment
2Wiki	0.70	10	8	6	1.25
	0.90	24	20	10	1.20
MuSiQue	0.50	16	12	8	1.33
	0.60	32	32	16	1.00
	0.70	128	96	48	1.33
HotpotQA	0.75	8	8	6	1.00
	0.90	32	32	20	1.00

5.4.5 Coverage and Target Budgets

At $B = 8$, unit-increment allocation raises complete-evidence coverage from 0.688 to 0.751 on 2Wiki and from 0.413 to 0.460 on MuSiQue. The corresponding segment-policy coverages are 0.746 and 0.445. On HotpotQA, unit-increment coverage is 0.770 against the balanced 0.762 at eight slots, and falls below balanced coverage at sixteen and thirty-two slots. Exact optimisation of the envelope thus has budget-dependent effects on measured union coverage. A position-weighted allocation reaches 0.437 on MuSiQue at $B = 8$, compared with the learned segment policy’s 0.445; its corresponding 2Wiki coverage is 0.625. These comparators help distinguish the benefit of predicting depth from that of using a simple uneven split.

The target budgets in Table 5.6 are first crossings on the tested grid, rather than interpolated minimum costs. For example, MuSiQue reaches 50% coverage at budget 12 under segment allocation and 16 under balanced allocation. This is a selected-slot ratio of 1.33 at that target. Its interpretation excludes the common trace-generation cost and makes no assumption about final-answer accuracy.

5.4.6 Paired Final-Answer Evaluation

The completed reader experiment takes the first 700 retained MuSiQue traces and refits the segment-policy allocator using five-fold question-level cross-validation within

those 700 cases. Its coverage values therefore refer to a different subset and fitted models from the 824-question frontier. Each budget–policy pair is evaluated on every selected question, giving 4 200 completed final-reader calls. These outcomes were generated before the unit-increment revision and measure the segment policy throughout.

Both policies deduplicate and interleave their selected prefixes. The same GPT-4o-mini reader, requested through the API’s model alias at temperature zero, answers the original question from that pooled context. Table 5.7 reports the paired outcomes.

Table 5.7: Pooled final-reader evaluation on 700 retained MuSiQue traces. U denotes balanced allocation and L the learned segment heuristic. All 4,200 question–budget–policy evaluations completed. The last column gives L minus U and its descriptive pointwise paired 95% F1 interval; the intervals at budgets 16 and 32 contain zero. The unit-increment allocator is a separate policy.

Budget	Cov. U	Cov. L	EM U	EM L	F1 U	F1 L	Δ F1 [95% CI]
8	0.413	0.449	0.344	0.354	0.424	0.445	+0.021 [0.004, 0.039]
16	0.523	0.547	0.360	0.377	0.454	0.464	+0.010 [−0.008, 0.027]
32	0.603	0.600	0.367	0.359	0.472	0.458	−0.014 [−0.031, 0.001]

At $B = 8$, segment allocation raises F1 from 0.4244 to 0.4455 and exact match from 0.3443 to 0.3543. The paired F1 difference is 0.0211, with a descriptive 95% interval of [0.0038, 0.0385] from 10 000 question-level bootstrap resamples. The corresponding F1 differences at $B = 16$ and $B = 32$ are 0.0098 and -0.0144 , with intervals [−0.0076, 0.0270] and [−0.0309, 0.0014]. These are pointwise intervals over the recorded pairs, without adjustment across budgets or separation of shared source components. They describe the development-panel comparison rather than independent confirmation.

Every allocation spends its declared prefix budget. Deduplication leaves mean context sizes of 7.37 versus 7.39 passages at $B = 8$, and 30.04 versus 30.31 at $B = 32$. Thus the matched quantity is selected slots, with unique passages and token lengths free to differ. The observed answer gain at the smaller budget concerns this pooled-context intervention. Changing an intermediate reading budget requires regeneration of its dependent queries to test a sequential execution policy.

5.4.7 Exploratory Hidden-State Prediction of Assigned Depth

Chapter 3 studies representations of compositional hop count. A separate probe examines their association with the assigned depth target d_a defined above. The panel contains 2 648 reference arms from 1 000 MuSiQue questions with BM25 rankings. Supporting passages are assigned to the earliest-ranking arm; 31 arm targets belong to questions whose required evidence is unreachable within the retained lists. The binary targets are $d_a \leq k$ for $k \in \{2, 4, 8, 16, 32\}$.

The frozen models encode the reference subquestion templates as raw text, without a chat template. Right-padding-aware last-token pooling supplies the features at nine sampled layers per model. Of the 2 648 templates, 1 517 contain unresolved #N references and differ from the gold-resolved queries used for retrieval. The representations therefore describe the templates rather than the executed queries.

Five-fold cross-validation keeps all arms of a question together. Feature standardisation is fitted within each training fold. Logistic regression uses inverse regularisation strength $C = 0.01$ for hidden states and $C = 1$ for the surface-and-score comparator,

Table 5.8: Exploratory depth prediction on the fixed reference panel. Mean AUROC averages the five depth thresholds. Each model’s layer is selected using the same cross-validation results reported here.

Features	Selected layer	Mean AUROC
Qwen2.5-7B-Instruct	14	0.8397
Llama-3.1-8B-Instruct	12	0.8454
Mistral-7B-Instruct-v0.3	16	0.8442
Surface features and top-ten BM25 scores	n/a	0.7639

so Table 5.8 compares two specified readouts as well as their features. Near-constant hidden-state dimensions are filtered using the full panel before splitting.

The split retains substantial source overlap: 1 557 test-arm instances have an identical template in their training fold, and 755 test questions share a source subproblem with training. Layer selection on the reported folds adds selection optimism. All selected-layer activations are finite; the Qwen final-layer arrays contain nonfinite activations for 2 237 arms, which the probe code replaces with zero. That layer is unsuitable for interpreting a layer-wise performance decline.

These measurements motivate a component-separated evaluation of template representations with nested layer selection and matched readout tuning. Their use in an allocator requires a further end-to-end comparison of evidence coverage, answer quality and representation-extraction cost.

5.5 Evidence-Aware Feedback for Prompt Search

5.5.1 Separating Reward from Diagnostic Information

A scalar reward ranks candidate instructions. A diagnostic record identifies features of their executions that the optimizer can use when proposing a revision. Keeping these roles separate allows the same final-answer reward to be paired with different feedback representations.

For a question q , the proposed record contains the generated plan, resolved queries, intermediate answers, retrieved evidence and final-answer score. The gold passages and reference decomposition are available to the optimisation-side feedback service. During execution, the decomposer and reader use the question, fixed demonstrations, retrieved passages and their own predicted bindings. Gold intermediate answers enter feedback after the execution rather than resolving its dependencies.

The implemented feedback builder supports three views:

1. **Answer feedback.** The question, generated plan, final answer, reference answers, exact match and F1.
2. **Raw evidence feedback.** Answer feedback plus reference obligations and their passage text, executed queries, intermediate predictions and passage ranks. Available cell votes from the frozen ownership ensemble are included as measurements.
3. **Structured evidence feedback.** The same raw evidence payload plus explicit dependency edges, per-obligation retrieval status, estimated ownership status and restricted binding checks.

Table 5.9: Diagnostic fields supplied by the structured feedback interface.

Field	Measurement	Scope
Dependency structure	Plan references and interface validity	A property of the generated plan
Reader evidence gap	Required passage absent from every five-passage reader context	An observation of this execution
Retained-list gap	Required passage absent from all saved rankings	Censored at the retained depth
Unassigned obligation	An obligation with zero estimated owners	An estimated assignment gap
Fold disagreement	Votes differ for an arm–evidence pair	Estimator uncertainty
Binding comparison	Intermediate answer compared with an aligned reference	Available only for a uniquely matched template and its dependencies

The raw and structured views share identical underlying evidence text and ownership measurements for a given trace. Their shared payload is hashed after common clipping, and both obey the same overall reflection input cap. Structured feedback adds a derived organisation of that information. Its comparison with raw feedback therefore tests the value of the representation, while the comparison with answer-only feedback also includes additional supervision. Actual reflection tokens remain a reported cost because equal caps do not imply equal realised input lengths.

5.5.2 Observed Events and Estimated Assignments

Table 5.9 distinguishes deterministic observations from classifier-dependent interpretations. The distinction prevents a failed answer from being assigned a single cause on the basis of a score alone.

The current binding check uses normalised exact subquestion templates and requires their referenced predecessors to align consistently. It leaves paraphrased, merged or otherwise ambiguous arms unmatched. A benchmark answer mismatch on an aligned arm is evidence about that intermediate prediction, not proof that it caused the final error. Causal attribution would require controlled re-execution that changes the suspected failure while holding the remaining policy fixed.

Ownership estimates retain the five model votes and the identity of the frozen estimator. A predicted unowned passage is described as a suspected unassigned obligation. When a passage is available in a different arm’s context, the record distinguishes global availability from the evidence available to its estimated owner. This is the distinction that the union and masked depths formalised in Chapter 4.

5.5.3 Reflective Search with a Fixed Execution Interface

GEPA supplies a reflective prompt-search procedure that uses execution feedback and candidate performance across examples [?]. The extension uses the pinned GEPA implementation to optimise the single decomposer instruction. The same adapter evalu-

ates answer, raw and structured feedback conditions. The proposed contribution is the evidence-feedback representation within this common search procedure.

The primary comparison uses final-answer F1 as its scalar reward in every condition. The original masked-depth score, complete-evidence coverage and resource use are retained as diagnostics. Candidate selection uses the instance-level Pareto strategy, with reflection batches of eight optimisation examples and strict improvement acceptance. Examples are sampled by shuffled epochs, and their exposure counts are logged.

Only one text component is optimised. The component-exchange merge operator is disabled, because it would not provide a substantive combination of different components in this setting. The experiment is consequently a single-component reflective mutation comparison. Calling it an evolutionary search does not imply that every possible mutation, crossover or weight-training method is evaluated.

The execution contract permits one to six arms and strictly backward references. Invalid plans trigger a shared direct-question fallback, which remains in the quality and cost denominators. The reader uses five passages per arm, and the same model settings, demonstrations and passage corpus are used across fresh search conditions. This matches the maximum execution allowance while leaving actual arm count and tokens free to differ.

Fresh task execution uses GPT-4o-mini, snapshot `gpt-4o-mini-2024-07-18`. Reflection uses GPT-4o, snapshot `gpt-4o-2024-08-06`. The runner checks both returned model identifiers. The declared task seed and first search seed are one, with temperature zero for execution and one for reflection. Provider identity and generation settings enter the cache and run fingerprints. These settings remain separate from the historical Llama-based pilot.

5.6 Independent Evaluation Design

5.6.1 Frozen Data Roles and Corpus Scope

The new comparison separates questions used for reflection, questions used for candidate selection and questions reserved for final confirmation. The source is the official answerable MuSiQue development release [4]. Excluding all composite question IDs in the historical thousand-question panel leaves 1417 cases. Removing three additional exact normalised question-text matches leaves 1414.

Of those questions, 363 have every gold supporting passage in the fixed 11 656-passage corpus. The initial matched-backbone study uses this eligible subset. Eligibility is determined before new model outputs, by exact passage identity rather than observed retrieval success. It defines a corpus-covered target population. Broader MuSiQue performance requires a separate evaluation with the additional corpus material included.

Questions sharing any underlying single-hop source ID are grouped before the new splits are assigned. The final manifests contain 96 optimisation, 48 selection and 200 confirmation questions, with zero shared source IDs across these three splits. Table 5.10 gives their composition. Split identities and file hashes are frozen before candidate generation.

All selected composite question IDs and texts differ from the historical panel and the ownership classifier’s training panel. Historical subproblem reuse remains: 343 of the 344 selected questions share at least one single-hop source with the historical thousand questions. These are new compositions under the documented exposure exclusions, rather than wholly new atomic facts. There is no source-ID overlap with the classifier’s training

Table 5.10: Frozen question-new splits for the prospective comparison. Components connect questions sharing a source single-hop ID.

Role	Questions	Two-hop	Three-hop	Four-hop	Components
Optimisation	96	23	40	33	29
Selection	48	16	19	13	22
Confirmation	200	73	77	50	65

questions. The confirmation set therefore tests transfer to new compositions within the corpus-covered regime.

The reflection service accepts only hash-bound optimisation samples. The selection set supplies scores for ranking candidates and remains adaptive development data. Confirmation question contents and labels stay outside the search interface until the final instructions and analysis choices have been fixed.

5.6.2 Comparators and Search Allowances

The fresh study includes the unchanged initial instruction, nonadaptive instruction sampling, OPRO-style answer-feedback search and three GEPA feedback conditions. Nonadaptive proposals are generated from the task and initial instruction before their evaluation results are available. The OPRO control uses score histories and answer errors. The three GEPA conditions share the optimizer, reward and execution interface while varying the feedback view.

Each fresh search is assigned a ceiling of 512 example-level rollout evaluations and 24 candidate proposals. A rollout is an executed candidate-question pair. Parent comparisons, proposed-candidate comparisons and validation executions count toward that ceiling. The different methods allocate their allowance differently, so search progress is measured against cumulative evaluations and actual cost.

The nonadaptive schedule screens 24 instructions on a common set of 12 optimisation questions, then evaluates its four leaders on the 48 selection questions. This uses

$$24 \times 12 + 4 \times 48 = 480 \quad (5.11)$$

candidate-question evaluations. The OPRO control uses four rounds of six proposals, with a declared twelve-question screen per round and selection evaluation for each round’s nominee. The reflective search uses its allowance for parent and child batches and complete selection evaluations. The number of completed mutations is reported explicitly rather than inferred from the evaluation ceiling.

Within a search condition, the winner is selected by mean selection F1. Ties are resolved by lower mean execution tokens and then instruction hash. The fixed initial instruction and the historical OPRO winner are retained as identified reference policies. The fresh pipeline may use a different API model configuration from the historical Llama pilot; comparisons are made within a frozen fresh backbone, and historical scores remain in their original regime.

The principal contrast is structured versus answer-only feedback. Structured versus raw feedback is required to isolate the effect of diagnostic organisation from additional supervision. A second predeclared search seed provides a replication of search variability when resources permit. Both seeds’ trajectories must then be retained.

5.6.3 Outcome Analysis and Cost Accounting

After selection, each final policy is evaluated on the same locked confirmation questions. The primary outcome is the question-weighted mean paired F1 difference. Exact match, complete evidence in the reader contexts, union depth, masked depth, arm count and model tokens are secondary measurements. All questions, including fallback executions, contribute to the aggregate results.

Paired bootstrap resampling preserves each question’s outcomes across policies. A complementary component-level bootstrap keeps questions sharing a source component together. The confirmation panel contains 65 such components, with at most 20 questions in one component. Question and component resampling describe sensitivity under different sampling units; neither supplies a distribution-free generalisation guarantee.

Search cost and inference cost are recorded separately. Search cost includes instruction proposals, reflection and candidate evaluation. Inference cost includes decomposition, retrieval-associated model calls and every reader step for the selected policy. Cached deterministic evaluations count toward the logical search allowance while their actual charges are recorded separately. A cache hit is reuse of a result, not an independent execution.

The passage-allocation replay in Chapter 4 provides a complementary cost diagnostic. On recorded lists it compares balanced total-slot allocation with the smallest oracle prefix allocation covering the gold passages. The oracle has access to gold identities and frozen rankings. It measures available allocation headroom; it is not a learned execution policy. In the sequential pipeline, changing an early reader allowance may change its answer and every downstream query. Such a policy requires fresh execution before claiming an answer-quality or runtime gain.

5.6.4 Separating Prompt and Allocation Effects

The completed studies intervene at different points: the OPRO pilot changes the decomposition instruction, while Section 5.4 changes prefixes selected from fixed rankings. A prospective factorial comparison can test their combination. Freeze an initial prompt p_0 , a search-selected prompt p_1 , balanced allocation g_0 and learned allocation g_1 before final evaluation. Evaluate all four pairs (p_i, g_j) on the same questions and reader budget. If Y_{ij} denotes mean final-answer F1, the interaction is

$$I = (Y_{11} - Y_{10}) - (Y_{01} - Y_{00}). \quad (5.12)$$

It asks whether the allocation benefit changes under the new prompt. Each prompt generates its own traces under the same five-passages-per-hop policy; both allocators then act on those traces. This comparison concerns pooled context, with trace-generation cost reported separately. Its outcomes remain prospective.

A sequential extension must choose each arm’s budget using information available at that step and reserve resources for later arms. It then recomputes intermediate answers and downstream retrieval. The offline allocator uses resolved later queries and their score profiles, which would give an online policy future information. Step-local allocation and re-execution are therefore part of that extension.

Table 5.11: Ownership predictions on the optimisation-only stress suite. Exact agreement requires every cell of a case’s ownership mask to match the transformation labels. Variants share twelve base questions.

Transformation	Cases	Exact masks
Reference plan	12	12
Duplicate relevant arm	12	12
Insert irrelevant arm	12	7
Omit sink arm	12	6
Fuse independent arms, gold-resolved	4	2

5.6.5 Controlled Checks of the Diagnostic Representation

A prepared stress suite examines how ownership diagnostics should respond to known plan transformations. Twelve optimisation questions are selected deterministically before classifier evaluation, with four at each of two, three and four hops and twelve distinct source components. They yield 52 cases: 12 reference plans, 12 relevant-arm duplications, 12 irrelevant-arm insertions, 12 sink-arm omissions and 4 independent-arm fusions.

Expected ownership masks are derived from transformation provenance. Duplication adds an owner for the same supporting passage. An irrelevant arithmetic query adds an arm with no obligation from the reference task. Omitting an unreferenced sink creates a missing obligation only when its supporting passage is unique among the remaining arms. Shared evidence is tracked separately so that deleting a hop is not automatically equated with deleting its passage coverage.

The four fusions combine independent reference arms whose distinct answers are subsequently used downstream. The scalar reference grammar provides no selector for multiple outputs from one fused arm. These four cases therefore use explicitly gold-resolved downstream inputs and test ownership on synthetic plans. They are excluded from claims about executable fused policies. The remaining transformations preserve valid backward references.

The frozen five-model ensemble was evaluated on all 52 cases using the reconstructed feature pipeline. Table 5.11 compares the predicted mask with the complete provenance-derived mask for each case. These transformation-suite measurements are separate from the historical classifier evaluation in Chapter 4.

Reference plans and relevant-arm duplications preserve the expected masks in all twelve cases. After sink omission, the ensemble identifies six of the twelve missing obligations. Irrelevant insertion and omission produce additional positive assignments, making missingness sensitive to the estimator. This supports describing an unowned obligation as suspected rather than treating the mask as an omission oracle. The suite measures response to controlled, reference-derived transformations; the variants are dependent and the fused cases retain their gold-resolved scope.

5.7 Discussion

5.7.1 What the Completed Experiment Establishes

The pilot uses the attribution method of Chapter 4 within an executable prompt search. It demonstrates that the measured instruction objective can change under au-

tomated proposals while the retrieval and reading backbone remains fixed. The selected instruction improves the masked-depth score by 0.90 points on the development panel, preserves aggregate finite masked coverage and produces similar final-answer quality. Its higher mean arm count makes the cost distinction explicit.

The candidate-level comparison also identifies a practical choice. Selecting for evidence depth and selecting for final-answer F1 can choose different instructions from the same search. An optimiser intended to improve QA should therefore be evaluated on the answer outcome, with the evidence measurements explaining where execution changes occur. An optimiser intended to improve retrieval depth should state that objective and measure total resource use alongside it.

The allocation study adds a measured intervention on the evidence selected from recorded rankings. Its coverage gains are largest on 2Wiki at the smaller tested budgets, with more limited differences on MuSiQue and HotpotQA. The unit-increment policy optimises the concave interpolant exactly. Prediction, assigned-depth targets, that interpolant and the arm floor still contribute to the measured policy. The gap to the gold-informed union oracle combines these factors. The paired segment-policy MuSiQue reader experiment shows a 2.1-point F1 increase at eight slots and a 1.4-point decrease at thirty-two slots. Allocation therefore needs assessment at the intended budget and on final answers, alongside the evidence-coverage objective.

5.7.2 The Scope of the Evidence-Feedback Extension

The extension turns the rank matrix and ownership estimates into structured records for prompt revision. Its design separates an observed reader-context gap, an estimated assignment gap and an aligned intermediate-answer mismatch. It also preserves an information-matched raw view, enabling a direct test of whether this organisation helps the optimizer.

Chapter 4 describes the ownership model’s benchmark evaluation. The stress suite adds controlled duplication, insertion, omission and fusion measurements. Automatically optimised instructions can generate other arm styles and relations, which require a generated-plan attribution audit. Keeping final-answer F1 as the primary reward reduces reliance on estimated ownership for winner selection, while preserving ownership as diagnostic feedback.

The fresh comparison is designed to establish performance of frozen instructions on new compositions. The fixed corpus eligibility filter, historical subproblem reuse and finite confirmation panel delimit its generalisability. Transfer to an expanded corpus, another dataset or another reader is a separate experiment. The chapter supplies completed prompt and allocation studies and a controlled protocol for measuring whether evidence feedback and allocation improve final-answer quality together.

Conclusion

This chapter optimised a decomposition instruction using the ownership-masked retrieval score introduced in Chapter 4. Four rounds produced twenty-four new instructions, evaluated alongside the seed on one hundred questions. The selected instruction increased the score from 66.30 to 67.20, while exact match remained 44% and F1 changed from 52.37 to 52.48. The distinction between retrieval depth, answer quality and total execution cost is therefore retained in the assessment of prompt changes.

A complementary learned allocator uses arm features and shallow score profiles to distribute a fixed passage-slot budget. On generated sequential BM25 traces, unit-increment allocation increases eight-slot coverage by 6.3 percentage points on 2Wiki, 4.7 on MuSiQue and 0.8 on HotpotQA. It optimises the concave-interpolant objective exactly; its measured coverage effects vary across budgets. Their cost accounting separates selected passages from the computation that generated the traces. On the earlier segment policy’s 700-question pooled-reader comparison, F1 rises from 0.424 to 0.445 at eight slots, with budget-dependent effects across the three tested settings.

The proposed continuation uses final-answer F1 for selection and evidence responsibility for diagnostic feedback. Its implemented interfaces separate raw observations from estimated assignments, and its frozen evaluation design compares answer-only, raw and structured feedback under common search and execution allowances. Together, Chapters 4 and 5 provide an evidence-aware measurement, its use in completed prompt and allocation studies, and a controlled design for testing their contribution to answer quality on new questions.

Conclusion and outlook

This internship project aimed to design

Bibliography

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 2369–2380, 2018.
- [3] X. Ho, A.-K. D. Nguyen, S. Sugawara, and A. Aizawa, “Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps,” in *Proceedings of the 28th International Conference on Computational Linguistics*, pp. 6609–6625, 2020.
- [4] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “MuSiQue: Multihop questions via single-hop question composition,” *Transactions of the Association for Computational Linguistics (TACL)*, vol. 10, pp. 539–554, 2022.
- [5] O. Abdelhedi, O. R. Heidari, S. Chaari, X. Chen Zhu, M. Hamed, D. Azzam, and Y. Yaakoubi, “Probing compositional question difficulty in language model representations,” in *IEEE AIDIST 2026 Conference*, 2026. Accepted, to appear.
- [6] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [7] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [8] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.
- [9] O. Khattab and M. Zaharia, “ColBERT: Efficient and effective passage search via contextualized late interaction over BERT,” in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 39–48, 2020.
- [10] W. Xiong, X. L. Li, S. Iyer, J. Du, P. Lewis, W. Y. Wang, Y. Mehdad, W.-t. Yih, S. Riedel, D. Kiela, and B. Oğuz, “Answering complex open-domain questions with

- multi-hop dense retrieval,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [11] O. Khattab, C. Potts, and M. Zaharia, “Baleen: Robust multi-hop reasoning at scale via condensed retrieval,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.
- [12] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10014–10037, 2023.
- [13] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically inspired long-term memory for large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [14] B. J. Gutiérrez, Y. Shu, W. Qi, S. Zhou, and Y. Su, “From RAG to memory: Non-parametric continual learning for large language models,” in *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267 of *Proceedings of Machine Learning Research*, pp. 21497–21515, PMLR, 2025.
- [15] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, “Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 7036–7050, 2024.
- [16] Z. Jiang, F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, “Active retrieval augmented generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 7969–7992, 2023.
- [17] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [18] G. Alain and Y. Bengio, “Understanding intermediate layers using linear classifier probes,” *arXiv preprint arXiv:1610.01644*, 2016.
- [19] K. Park, Y. J. Choe, and V. Veitch, “The linear representation hypothesis and the geometry of large language models,” in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235 of *Proceedings of Machine Learning Research*, pp. 39643–39666, PMLR, 2024.
- [20] J. Hewitt and P. Liang, “Designing and interpreting probes with control tasks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 2733–2743, Association for Computational Linguistics, 2019.
- [21] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez, N. Schiefer, Z. Hatfield-Dodds, N. DasSarma, E. Tran-Johnson, S. Johnston, S. El-Showk, A. Jones, N. Elhage, T. Hume, A. Chen, Y. Bai, S. Bowman, S. Fort, D. Ganguli,

- D. Hernandez, J. Jacobson, J. Kernion, S. Kravec, L. Lovitt, K. Ndousse, C. Olsson, S. Ringer, D. Amodei, T. Brown, J. Clark, N. Joseph, B. Mann, S. McCandlish, C. Olah, and J. Kaplan, “Language models (mostly) know what they know,” *arXiv preprint arXiv:2207.05221*, 2022.
- [22] C. Burns, H. Ye, D. Klein, and J. Steinhardt, “Discovering latent knowledge in language models without supervision,” in *The Eleventh International Conference on Learning Representations*, 2023.
- [23] S. Marks and M. Tegmark, “The geometry of truth: Emergent linear structure in large language model representations of true/false datasets,” in *First Conference on Language Modeling*, 2024.
- [24] A. Azaria and T. Mitchell, “The internal state of an LLM knows when it’s lying,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 967–976, Association for Computational Linguistics, 2023.
- [25] I. V. Moreno Cencerrado, A. Padrés Masdemont, A. Gonzalvez Hawthorne, D. D. Africa, and L. Pacchiardi, “No answer needed: Predicting LLM answer accuracy from question-only linear probes,” *arXiv preprint arXiv:2509.10625*, 2025.
- [26] S. Yang, E. Gribovskaya, N. Kassner, M. Geva, and S. Riedel, “Do large language models latently perform multi-hop reasoning?,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10210–10229, 2024.
- [27] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, “Measuring and narrowing the compositionality gap in language models,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 5687–5711, 2023.
- [28] S. Min, V. Zhong, L. Zettlemoyer, and H. Hajishirzi, “Multi-hop reading comprehension through question decomposition and rescoring,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 6097–6109, 2019.
- [29] E. Perez, P. Lewis, W.-t. Yih, K. Cho, and D. Kiela, “Unsupervised question decomposition for question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 8864–8880, 2020.
- [30] D. Dua, S. Gupta, S. Singh, and M. Gardner, “Successive prompting for decomposing complex questions,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 1251–1265, 2022.
- [31] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. V. Le, and E. H. Chi, “Least-to-most prompting enables complex reasoning in large language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [32] T. Khot, H. Trivedi, M. Finlayson, Y. Fu, K. Richardson, P. Clark, and A. Sabharwal, “Decomposed prompting: A modular approach for solving complex tasks,” in *International Conference on Learning Representations (ICLR)*, 2023.

Netography

- [N1] Concordia University, “The birth of Concordia: Loyola College and Sir George Williams University merged on 24 August 1974.” <https://www.concordia.ca/cunews/main/stories/2015/08/24/archives-loyola-and-sir-george-williams-merge.html>. Accessed: 2026-08-13.
- [N2] Concordia University, “Sir George Williams University history.” <https://www.concordia.ca/offices/archives/stories/sgw.html>. Accessed: 2026-08-13.
- [N3] Concordia University, “Concordia fast facts: student enrolment 2024–25.” <https://www.concordia.ca/about/fast-facts.html>. Accessed: 2026-08-13.
- [N4] Concordia University, “First engineering faculty in Canada named after a woman: business leader Gina Cody makes historic \$15 million donation.” <https://www.concordia.ca/news/stories/2018/09/24/gina-cody-makes-historic-eponymous-15m-donation-to-have-canadas-first-engineering-faculty-named-after-a-woman.html>. Published 24 September 2018. Accessed: 2026-08-13.
- [N5] Concordia University, “Gina Cody — notable Concordians.” <https://www.concordia.ca/profiles/offices/advancement/notable-concordians/gina-cody.html>. Accessed: 2026-08-13.
- [N6] Concordia University, “About the school — gina cody school of engineering and computer science.” <https://www.concordia.ca/ginacody/about.html>. Accessed: 2026-08-13.